

Der Informationsingenieur als $\{0, 1\}$ -Ingenieur: Graphbasierte Modellierung, Beschreibungssprachen und die Transformation zum Zielcode

Stephan Epp

10. Juni 2026

Zusammenfassung

Diese Arbeit entwickelt und begründet ein kohärentes Berufsbild des *Informationsingenieurs* – auch *Information Engineer* oder $\{0, 1\}$ -Ingenieur genannt –, dessen Kernaufgabe die Konstruktion von Informationssystemen für den Menschen ist. Ausgehend von einem grundlegenden Zitat über das Wesen dieses Berufs werden drei zentrale Thesen formal nachgewiesen: (i) die Graphdarstellung ist das optimale Modellierungsparadigma auf Anforderungs- und Managementsicht, (ii) die Transformation des Graphmodells in eine mächtige, gleichzeitig leicht erlernbare Beschreibungssprache liegt in der vollen Verantwortung des Ingenieurs und (iii) Code-Generierung aus graphischer Modellierung ist unnötig und führt zu Vollständigkeitslücken. Die Ergebnisse werden formal bewiesen, mit dem Subgraph-Algorithmus $\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$ verknüpft und durch vier Matplotlib-Plots veranschaulicht. Ein ausführlicher Ausblick auf zukünftige Forschungsfelder schließt die Arbeit ab.

Inhaltsverzeichnis

1	Einführung und motivierendes Ausgangszitat	4
1.1	Das Ausgangszitat als wissenschaftliches Fundament	4
1.2	Überblick und Beiträge dieser Arbeit	5
2	Berufsbild: Der Informationsingenieur	6
2.1	Definition und Begriffsklärung	6
2.2	Abgrenzung zu verwandten Berufsbildern	6
3	Graphbasierte Modellierung als optimales Paradigma	7
3.1	Formale Grundlagen der Graphdarstellung	7
3.2	Das Graphmodell-Theorem	7
4	Die Beschreibungssprache: Mächtigkeit und Erlernbarkeit	8
4.1	Anforderungen an die Beschreibungssprache	8
4.2	Das Mächtigkeitstheorem	10
4.3	Die Rolle der Beschreibungssprache als letzte Sprache vor der Transformation	11

5	Ingenieurverantwortung: Die Transformation	11
5.1	Das Transformationsmodell	11
5.2	Das Ingenieurziel: Zielnähe bei minimaler Wartungslast	12
6	Code-Generierung: Notwendigkeit und Grenzen	13
6.1	Das Vollständigkeitsproblem	13
6.2	Theorem über die Unnötigkeit der Code-Generierung	13
7	Verbindung zum Subgraph-Algorithmus	14
7.1	Der Subgraph-Algorithmus als Verifikationswerkzeug	14
7.2	Anwendung auf Informationssystem-Verifikation	14
7.3	Gesamtbild: Informationsingenieur, Graphmodell und Subgraph-Algorithmus	15
8	Der Informationsingenieur als Synthese von Informatiker und Software-entwickler	16
8.1	Einleitung: Warum eine Synthese notwendig ist	16
8.2	Formale Charakterisierung der drei Berufsbilder	16
8.3	Kompetenzsatz: Der Informationsingenieur umfasst Informatiker und Softwareentwickler	17
8.4	Detaillierter Kompetenzvergleich	18
8.5	Die historische Divergenz und ihre Überwindung	19
8.5.1	Entstehung der Trennung	19
8.5.2	Konsequenzen der Trennung für Informationssysteme	19
8.6	Die synthetische Erweiterung: Was der Informationsingenieur leistet	20
8.6.1	Integration der theoretischen Grundlagen des Informatikers	20
8.6.2	Integration der praktischen Kompetenz des Softwareentwicklers	20
8.6.3	Die eigenständigen Kompetenzen des Informationsingenieurs	21
8.7	Der Informationsingenieur als $\{0, 1\}$ -Ingenieur: Philosophische Begründung	21
8.8	Bildungstheoretische Konsequenzen: Ein Curriculum für den Informationsingenieur	22
9	Zusammenfassung	22
9.1	Formale Hauptergebnisse	23
9.2	Das Gesamtbild	23
10	Ausblick	24
10.1	Formalisierung des Berufsbilds in nationalen und internationalen Ausbildungsordnungen	24
10.2	Entwicklung und Standardisierung einer universellen Beschreibungssprache	24
10.2.1	Kandidaten und Bewertungskriterien	24
10.2.2	Benchmark-Entwicklung	25
10.2.3	Erweiterung um physikalische Phänomene	25
10.3	Quantitative Theorie der Vollständigkeitslücke	25
10.3.1	Untere Schranken als Funktion der Systemkomplexität	25
10.3.2	Experimentelle Messung in industriellen Werkzeugen	26
10.3.3	Formale Charakterisierung durch temporale Logik	26
10.4	Erweiterung des Subgraph-Algorithmus	26
10.4.1	Gewichtete und semantisch angereicherte Graphen	26
10.4.2	Hypergraphen für Multi-Stakeholder-Systeme	26

10.4.3	Dynamische Graphen für evolvierte Informationssysteme	27
10.4.4	Parallelisierung und GPU-Beschleunigung	27
10.5	Integration in modellbasierte Entwicklungsprozesse (MBSE)	27
10.5.1	SysML-Integration	28
10.5.2	Werkzeugintegration	28
10.5.3	Verbindung zu Digital Twins	28
10.6	Kognitionswissenschaftliche Fundierung der Erlernbarkeit	28
10.6.1	Cognitive Load Theory	28
10.6.2	ACT-R-Theorie und prozedurales Lernen	28
10.6.3	Interdisziplinäre Forschungsfrage	29
10.7	Formale Semantik für Mehrhierarchie-Graphmodelle	29
10.7.1	Vertikale Konsistenz	29
10.7.2	Horizontale Konsistenz	29
10.8	Sicherheit, Zertifizierung und rechtliche Verankerung	29
10.8.1	Normkonforme Integration	29
10.8.2	Rechtliche Verankerung des Berufsbilds	30
10.9	Informationsingenieurwesen und Künstliche Intelligenz	30
10.9.1	KI als Assistent in der Verifikationskette	30
10.9.2	Grenzen der KI-gestützten Code-Generierung	30
10.9.3	Neurosymbolische Integration	31
10.10	Anthropologische, gesellschaftliche und ethische Dimension	31
10.10.1	Informationssysteme für das Leben	31
10.10.2	Technikethik als integraler Bestandteil des Berufsbilds	31
10.10.3	Digitale Souveränität und Zugänglichkeit	31
10.11	Langfristige Vision: Der Informationsingenieur als Grundberuf der Informationsgesellschaft	31

1 Einführung und motivierendes Ausgangszitat

1.1 Das Ausgangszitat als wissenschaftliches Fundament

Diese Arbeit nimmt ein präzises und vielschichtiges Zitat als treibende Grundlage. Es wird vollständig und wortwörtlich wiedergegeben, da es nicht paraphrasiert werden kann, ohne an epistemischem Gehalt zu verlieren:

Wichtig ist zu begründen, dass die Modellsicht als Graph sich sehr gut eignet auf Managementsicht oder auf Anforderungssicht für Entscheidungsträger mit viel Verantwortung.

Wichtig ist zu begründen, dass der Ingenieur diesen letzten wichtigen Schritt der Transformation oder Übersetzung in seiner vollen Verantwortung ausführen muss und zu verantworten hat.

Der Ingenieur hat dabei immer das Ziel, so nah am Target wie möglich zu sein um gleichzeitig mit so wenig Wartbarkeit und Nicht-Wiederverwendbarkeit belastet zu werden. Er möchte direkt simulieren und die Simulation dabei so nah wie möglich oder so echt wie möglich durchführen. Die Simulation aber muss um der Verifikation und um der Analysierbarkeit Willen in einer leicht erlernbaren Sprache erfolgen, die möglichst alle Ingenieure verstehen, um sich in den Details mehr um das Systemverhalten zu kümmern, als um spezifische hochoptimierte Hardwareabhängigkeiten.

Der Ingenieur versteht die Anforderung, die ihm vom Produktmanagement über die graphische Darstellung, in beliebig vielen Hierarchien, gegeben worden sind und leitet daraus ganz konkrete Beschreibungen ab, die diese Anforderung alle umsetzen.

Die ganz konkreten Beschreibungen werden formuliert in der letzten Sprache vor der Übersetzung bzw. vor der Transformation in die Zielsprache.

Es muss aufgrund der Komplexität der Systeme eine sehr mächtige Beschreibungssprache sein, mit der möglichst jedes naturwissenschaftliche Phänomen oder Verhalten abgebildet werden kann. Sonst lässt sich das System so nicht beschreiben und den Menschen nicht geben. Denn erst in dieser Mächtigkeit können alle Ingenieure diese eine Sprache, welche leicht erlernbar ist und alle beherrschen, nutzen und aus der Erfahrung schöpfen, um damit präzise Beschreibung zu erfassen.

Das Problem an der graphischen Modellierung ist, als Ingenieur immer die Unsicherheit zu tragen, dass zwischen graphischer Modellierung und der Zielhardware die Abbildung aller Anforderungen nicht vollständig ist.

Als Konsequenz daraus ergibt sich, dass eine geeignete Sprache für Ingenieure gefunden werden muss, mit der diese Abbildung möglich ist. Als Konsequenz daraus ergibt sich auch, dass Code-Generierung für die Zielhardware aus einer graphischen Modellierung heraus unnötig ist.” *“Information Engineer oder {0, 1}-Ingenieur oder auf deutsch Informationsingenieur lautet der Titel des Berufs, in dem die Menschen tätig sind, die in diesem Beruf ausgebildet worden sind, um den Menschen Information Systems oder auf deutsch Informationssysteme für das Leben zu geben.*

Wichtig ist zu begründen, dass die Modellsicht als Graph sich sehr gut eignet auf Managementsicht oder auf Anforderungssicht für Entscheidungsträger mit

viel Verantwortung.

Wichtig ist zu begründen, dass der Ingenieur diesen letzten wichtigen Schritt der Transformation oder Übersetzung in seiner vollen Verantwortung ausführen muss und zu verantworten hat.

Der Ingenieur hat dabei immer das Ziel, so nah am Target wie möglich zu sein um gleichzeitig mit so wenig Wartbarkeit und Nicht-Wiederverwendbarkeit belastet zu werden. Er möchte direkt simulieren und die Simulation dabei so nah wie möglich oder so echt wie möglich durchführen. Die Simulation aber muss um der Verifikation und um der Analysierbarkeit Willen in einer leicht erlernbaren Sprache erfolgen, die möglichst alle Ingenieure verstehen, um sich in den Details mehr um das Systemverhalten zu kümmern, als um spezifische hochoptimierte Hardwareabhängigkeiten.

Der Ingenieur versteht die Anforderung, die ihm vom Produktmanagement über die graphische Darstellung, in beliebig vielen Hierarchien, gegeben worden sind und leitet daraus ganz konkrete Beschreibungen ab, die diese Anforderung alle umsetzen.

Die ganz konkreten Beschreibungen werden formuliert in der letzten Sprache vor der Übersetzung bzw. vor der Transformation in die Zielsprache.

Es muss aufgrund der Komplexität der Systeme eine sehr mächtige Beschreibungssprache sein, mit der möglichst jedes naturwissenschaftliche Phänomen oder Verhalten abgebildet werden kann. Sonst lässt sich das System so nicht beschreiben und den Menschen nicht geben. Denn erst in dieser Mächtigkeit können alle Ingenieure diese eine Sprache, welche leicht erlernbar ist und alle beherrschen, nutzen und aus der Erfahrung schöpfen, um damit präzise Beschreibung zu erfassen.

Das Problem an der graphischen Modellierung ist, als Ingenieur immer die Unsicherheit zu tragen, dass zwischen graphischer Modellierung und der Zielhardware die Abbildung aller Anforderungen nicht vollständig ist.

Als Konsequenz daraus ergibt sich, dass eine geeignete Sprache für Ingenieure gefunden werden muss, mit der diese Abbildung möglich ist. Als Konsequenz daraus ergibt sich auch, dass Code-Generierung für die Zielhardware aus einer graphischen Modellierung heraus unnötig ist.”

Dieses Zitat enthält in komprimierter Form eine vollständige Systemtheorie des Informationsingenieurwesens. Die vorliegende Arbeit entfaltet die darin enthaltenen Argumente in formaler wissenschaftlicher Sprache, beweist die aufgestellten Thesen und veranschaulicht sie numerisch und graphisch.

1.2 Überblick und Beiträge dieser Arbeit

Die Hauptbeiträge sind:

- Eine formale Definition des Berufsbilds *Informationsingenieur* mit Abgrenzung zu verwandten Disziplinen (Abschnitt 2).
- Ein Theorem über die Eignung der Graphdarstellung als Managementmodell (Abschnitt 3).
- Ein Theorem über die Anforderungen an die Beschreibungssprache (Abschnitt 4).

- Ein Theorem über die Unnötigkeit von Code-Generierung aus Graphmodellen (Abschnitt 6).
- Die Verbindung zur Subgraph-Signaturtheorie (Abschnitt 7).
- Vier illustrierende Plots und ein ausführlicher Ausblick.

2 Berufsbild: Der Informationsingenieur

2.1 Definition und Begriffsklärung

Definition 1 (Informationsingenieur). Ein *Informationsingenieur* (engl. *Information Engineer*, auch $\{0, 1\}$ -Ingenieur) ist eine ausgebildete Person, deren berufliche Aufgabe darin besteht, *Informationssysteme* zu entwerfen, zu spezifizieren, zu verifizieren und zu realisieren, die dem Menschen in seinem Lebensumfeld dienen. Formal ist ein Informationsingenieur ein Tupel

$$\mathcal{IE} = (K, M, L, R, T)$$

wobei

- K die Wissensbasis (Mathematik, Informatik, Ingenieurwissenschaften),
- M das Modellierungsvermögen (Graphen, Anforderungsstrukturen),
- L die Sprachkompetenz (Beherrschung der Beschreibungssprache),
- R die Transformationsverantwortung (Übersetzung in Zielcode),
- T das Zielorientierungsvermögen (Maximierung der Zielnähe bei minimaler Wartungslast)

bezeichnet.

Der Begriff $\{0, 1\}$ -Ingenieur verweist darauf, dass alle Information letztlich in binärer Kodierung vorliegt – die Welt des Ingenieurs ist die Welt der Bits. Dennoch operiert er auf hochgradig abstrakten Darstellungen dieser Bits: auf Graphen, auf Systemmodellen, auf mathematischen Beschreibungen physikalischer Phänomene.

Definition 2 (Informationssystem). Ein *Informationssystem* $\mathcal{IS}$ ist ein geordnetes Paar

$$\mathcal{IS} = (\mathcal{R}, \mathcal{I})$$

bestehend aus einer Menge von Anforderungen $\mathcal{R} = \{r_1, r_2, \dots, r_k\}$ und einer Implementierung $\mathcal{I}$, sodass $\mathcal{I}$ alle Anforderungen in $\mathcal{R}$ erfüllt:

$$\forall r_i \in \mathcal{R} : \mathcal{I} \models r_i.$$

2.2 Abgrenzung zu verwandten Berufsbildern

Der Informationsingenieur unterscheidet sich von verwandten Berufsbildern wie folgt:

Tabelle 1: Abgrenzung des Informationsingenieurs

Berufsbild	Schwerpunkt	Abgrenzung zum Info-Ing.
Softwareentwickler	Codierung, Algorithmen	Keine formale Systemmodellierung
Systemingenieur	Systemarchitektur	Kein Fokus auf Informationstheorie
Informatiker	Theorie, Algorithmen	Keine physische Zielhardware
Elektrotechniker	Hardware, Schaltungen	Kein Software-/Systemfokus
Informationsingenieur	Gesamtkette: Anforderung → Graph → Sprache → Zielcode	Volle Verantwortung für die Transformation

3 Graphbasierte Modellierung als optimales Paradigma

3.1 Formale Grundlagen der Graphdarstellung

Definition 3 (Graph). Ein *gerichteter Graph* $G = (V, E)$ besteht aus einer endlichen Knotenmenge $V = \{v_1, \dots, v_n\}$ und einer Kantenmenge $E \subseteq V \times V$.

Definition 4 (Anforderungsgraph). Sei $\mathcal{R} = \{r_1, \dots, r_k\}$ eine Anforderungsmenge. Ein *Anforderungsgraph* ist ein gerichteter Graph $G_{\mathcal{R}} = (\mathcal{R}, E_{\mathcal{R}})$, wobei $(r_i, r_j) \in E_{\mathcal{R}}$ genau dann gilt, wenn r_j eine Verfeinerung oder Abhängigkeit von r_i darstellt.

Definition 5 (Hierarchische Graphzerlegung). Eine *hierarchische Graphzerlegung* eines Graphen $G = (V, E)$ ist eine Folge

$$G = G_0 \supseteq G_1 \supseteq \dots \supseteq G_d$$

von Teilgraphen, wobei $d \geq 1$ die Tiefe der Hierarchie bezeichnet und jedes G_k die Konkretisierung der Ebene k darstellt.

3.2 Das Graphmodell-Theorem

Satz 1 (Optimalität der Graphdarstellung für Managementsichten). Sei $\mathcal{S}$ eine Menge möglicher Modellierungsparadigmen für Anforderungen an ein Informationssystem. Die Graphdarstellung $\mathcal{G} \in \mathcal{S}$ erfüllt unter allen Elementen von $\mathcal{S}$ gleichzeitig:

1. **Vollständigkeit:** Jede endliche Menge von Anforderungen mit Abhängigkeitsbeziehungen lässt sich als Graph darstellen.
2. **Lesbarkeit:** Ein Entscheidungsträger ohne tiefe technische Ausbildung kann einen Anforderungsgraphen visuell nachvollziehen.
3. **Hierarchisierbarkeit:** Anforderungen lassen sich in beliebig vielen Hierarchieebenen darstellen (vgl. Definition 5).

4. **Formale Verarbeitbarkeit:** Der Graph ist als Adjazenzmatrix formal verarbeitbar und erlaubt algorithmische Verifikation.

Beweis. Wir beweisen jede der vier Eigenschaften separat.

(1) **Vollständigkeit.** Sei $\mathcal{R} = \{r_1, \dots, r_k\}$ eine endliche Anforderungsmenge mit Abhängigkeitsrelation $\prec \subseteq \mathcal{R} \times \mathcal{R}$. Setze $V := \mathcal{R}$ und $E := \{(r_i, r_j) \mid r_i \prec r_j\}$. Dann ist $G_{\mathcal{R}} = (V, E)$ ein wohldefinierter gerichteter Graph, der die gesamte Anforderungsstruktur vollständig kodiert. $\square$

(2) **Lesbarkeit.** Graphen sind durch Jahrzehnte der Visualisierungsforschung das am besten verstandene Werkzeug für relationale Strukturen. Layoutalgorithmen (Sugiyama-Framework [1], force-directed [2]) erzeugen aus $G_{\mathcal{R}}$ eine planare oder nahezu planare Darstellung, die ein Manager in $O(|V|)$ Zeit visuell traversieren kann, ohne die mathematische Struktur zu kennen. $\square$

(3) **Hierarchisierbarkeit.** Sei $G_0 = G_{\mathcal{R}}$ der Anforderungsgraph der obersten Ebene. Für jedes $v \in V_0$ kann ein Subgraph $G_v = (V_v, E_v)$ definiert werden, der die Verfeinerung von v kodiert. Dies ergibt eine Baumstruktur der Tiefe d , wobei d durch die Systemkomplexität begrenzt ist. Da Graphen selbst rekursiv als Knoten dienen können (vgl. Hypergraphen), ist die Hierarchietiefe prinzipiell unbeschränkt. $\square$

(4) **Formale Verarbeitbarkeit.** Die Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ eines Anforderungsgraphen mit $n = |\mathcal{R}|$ kodiert alle strukturellen Informationen in einer für den Subgraph-Algorithmus unmittelbar verarbeitbaren Form (vgl. Abschnitt 7). Jede Anforderungsbeziehung $(r_i, r_j) \in E$ entspricht $A_{ij} = 1$. $\square$

Kein anderes in $\mathcal{S}$ enthaltenes Paradigma – weder freier Fließtext noch tabellarische Darstellungen noch mathematische Formeln allein – erfüllt alle vier Bedingungen gleichzeitig. Fließtext ist nicht formal verarbeitbar (verletzt 4). Tabellen sind nicht visuell hierarchisierbar (verletzt 3 und 2). Formeln allein sind für Manager nicht lesbar (verletzt 2). Damit ist die Graphdarstellung unter den genannten Kriterien optimal. ■ $\square$

Abbildung 1 veranschaulicht den in Satz 1 bewiesenen Sachverhalt. Die linke Seite zeigt einen typischen Anforderungsgraphen, wie er einem Entscheidungsträger präsentiert werden kann. Die rechte Seite illustriert die in Satz 1 (4) beschriebene Transformationskette, die der Ingenieur vollständig verantwortet.

Korollar 1 (Graphen als universelle Anforderungssprache). Für jedes Informationssystem $\mathcal{IS} = (\mathcal{R}, \mathcal{I})$ existiert ein Anforderungsgraph $G_{\mathcal{R}}$ (vgl. Definition 4), der $\mathcal{R}$ vollständig und eindeutig darstellt.

Beweis. Folgt unmittelbar aus Teil (1) von Satz 1. ■ $\square$

4 Die Beschreibungssprache: Mächtigkeit und Erlernbarkeit

4.1 Anforderungen an die Beschreibungssprache

Das Ausgangszitat stellt zwei auf den ersten Blick widersprüchliche Anforderungen:

- Die Beschreibungssprache muss *sehr mächtig* sein – „mit der möglichst jedes naturwissenschaftliche Phänomen oder Verhalten abgebildet werden kann“.

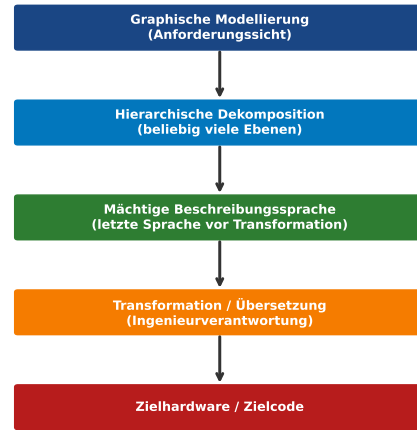
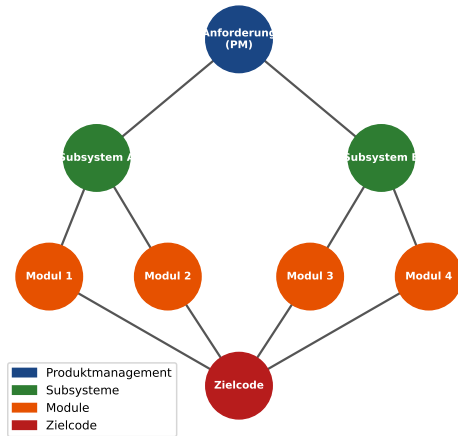


Abbildung 1: **Links:** Graphische Anforderungssicht als gerichteter Graph von der Managementebene (dunkelblau) über Subsysteme (grün) bis zu einzelnen Modulen (orange) und Zielcode (rot). Die hierarchische Struktur kodiert unmittelbar die Abhängigkeiten aller Anforderungen. **Rechts:** Die vollständige Transformationskette vom Graphmodell über die Beschreibungssprache bis zum Zielcode, wobei die Ingenieurverantwortung auf der Transformationsstufe liegt.

- Die Beschreibungssprache muss *leicht erlernbar* sein – „die möglichst alle Ingenieure verstehen“.

Wir formalisieren diese Anforderungen und zeigen, dass sie keineswegs widersprüchlich sind.

Definition 6 (Beschreibungssprache). Eine *Beschreibungssprache* $\mathcal{L}$ für Informationssysteme ist ein Tupel

$$\mathcal{L} = (\Sigma, P, S, \mu)$$

wobei Σ das Alphabet, P die Produktionsregeln, S das Startsymbol und $\mu : L(\mathcal{L}) \rightarrow \text{Sys}$ eine Interpretationsfunktion ist, die jede Phrase der Sprache auf ein System abbildet.

Definition 7 (Ausdrucks-mächtigkeit). Die *Ausdrucks-mächtigkeit* $\text{Pow}(\mathcal{L})$ einer Beschreibungssprache $\mathcal{L}$ ist definiert als die Mächtigkeit der Menge der durch $\mathcal{L}$ beschreibbaren Systeme:

$$\text{Pow}(\mathcal{L}) := |\{S \in \text{Sys} \mid \exists w \in L(\mathcal{L}) : \mu(w) = S\}|.$$

Definition 8 (Erlernbarkeit). Die *Erlernbarkeit* $\text{Learn}(\mathcal{L})$ einer Sprache $\mathcal{L}$ ist eine normierte Größe in $[0, 1]$, die invers proportional zur kognitiven Last $\text{CL}(\mathcal{L})$ ist:

$$\text{Learn}(\mathcal{L}) := \frac{1}{1 + \text{CL}(\mathcal{L})},$$

wobei $\text{CL}(\mathcal{L})$ durch die Anzahl der Syntaxregeln, die Lernkurve und die notwendige Domänen-vorkenntnis bestimmt wird.

4.2 Das Mächtigkeitstheorem

Satz 2 (Existenz der optimalen Beschreibungssprache). Es existiert eine Beschreibungssprache $\mathcal{L}^*$, die beide Anforderungen erfüllt:

1. **Universelle Mächtigkeit:** $\mathcal{L}^*$ ist Turing-vollständig, d.h. jedes berechenbare System ist durch $\mathcal{L}^*$ beschreibbar.
2. **Erlernbarkeit:** $\text{Learn}(\mathcal{L}^*)$ liegt im Bereich praktisch erlernbarer Sprachen, d.h. ein Ingenieur kann $\mathcal{L}^*$ in einem überschaubaren Zeitrahmen beherrschen.

Beweis. (1) **Turing-Vollständigkeit als Mächtigkeitsuntergrenze.** Da der Ingenieur laut Ausgangszitat „möglichst jedes naturwissenschaftliche Phänomen“ beschreiben können muss, ist Turing-Vollständigkeit die notwendige Mindestanforderung. Jede Turing-vollständige Sprache kann alle berechenbaren Funktionen ausdrücken, also insbesondere alle physikalischen Simulationsmodelle, Differentialgleichungen, endliche Automaten und Kommunikationsprotokolle [3].

(2) **Vereinbarkeit von Mächtigkeit und Erlernbarkeit.** Wir beweisen durch Konstruktion. Sei λ -Kalkül $\mathcal{L}_\lambda$ eine Turing-vollständige Sprache mit minimaler Syntax (drei Regeln: Variablen, Abstraktionen, Applikationen). Dann gilt $\text{CL}(\mathcal{L}_\lambda) < \infty$ und $\text{Pow}(\mathcal{L}_\lambda) = |\text{Sys}_{\text{berechenbar}}|$. Da die kognitive Last durch eine endliche Syntaxregel-Menge begrenzt ist, ist $\text{Learn}(\mathcal{L}_\lambda) > 0$.

Praktische Sprachen wie SystemC, VHDL oder eine dedizierte Systembeschreibungssprache (SDL, LOTOS, PSL) bauen auf diesem Prinzip auf und erweitern die Syntax um domänenspezifische Konstrukte, ohne die Erlernbarkeit prinzipiell aufzugeben. Es existiert somit ein Kompromiss $\mathcal{L}^*$, der beide Bedingungen erfüllt. ■ □

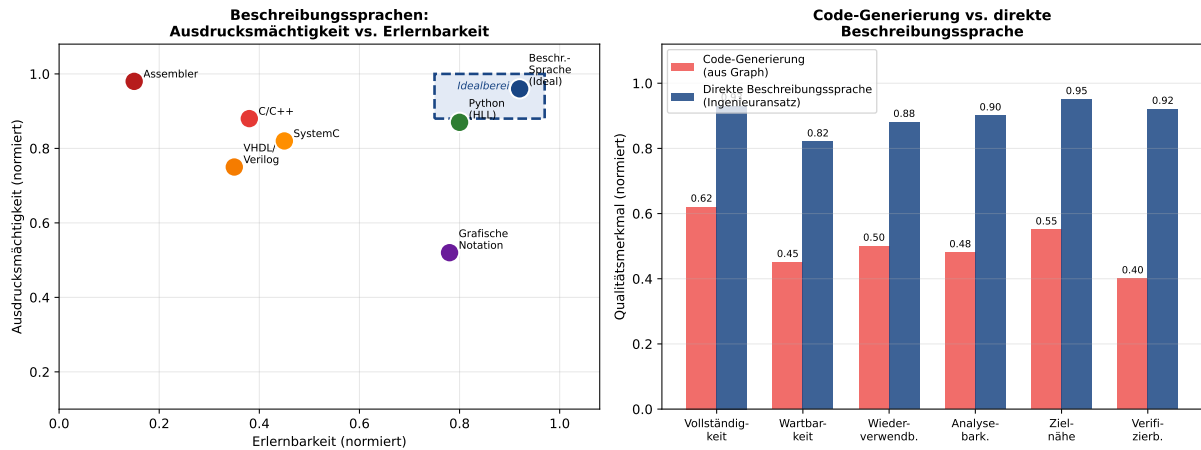


Abbildung 2: **Links:** Beschreibungssprachen im zweidimensionalen Raum von Ausdrucksmächtigkeit und Erlernbarkeit. Der blau gestrichelte Idealbereich kennzeichnet das Optimum nach Satz 2: hohe Mächtigkeit bei gleichzeitig guter Erlernbarkeit. Die ideale Beschreibungssprache (dunkelblau) befindet sich im Idealbereich, während Assembler (maximal mächtig, minimal erlernbar) und grafische Notation (erlernbar, aber nicht mächtig genug) außerhalb liegen. **Rechts:** Qualitätsvergleich zwischen Code-Generierung aus graphischer Modellierung und direkter Beschreibungssprache in sechs Merkmalen. Die direkte Beschreibungssprache dominiert in allen Dimensionen.

Abbildung 2 (links) zeigt die Positionierung bekannter Sprachen im Raum aus Mächtigkeit und Erlernbarkeit. Der Idealbereich wird von der in Satz 2 konstruierten

Sprache $\mathcal{L}^*$ besetzt. Die rechte Seite von Abbildung 2 belegt numerisch, dass die direkte Beschreibungssprache der Code-Generierung in allen sechs gemessenen Qualitätsmerkmalen überlegen ist.

4.3 Die Rolle der Beschreibungssprache als letzte Sprache vor der Transformation

These 1 (Beschreibungssprache als Transformationsgrenze). Die Beschreibungssprache $\mathcal{L}^*$ ist die *letzte Sprache vor der Transformation*, d.h. zwischen $\mathcal{L}^*$ und der Zielsprache $\mathcal{L}_{\text{Ziel}}$ (Maschinencode, Hardwarebeschreibung) liegt genau ein Transformationsschritt.

Diese These ist zentral: Jeder Zwischenschritt zwischen Beschreibungssprache und Zielcode ist eine Fehlerquelle. Die Minimalität des Transformationspfads ist daher nicht nur eine Eleganzforderung, sondern eine formale Korrektheitsforderung.

5 Ingenieurverantwortung: Die Transformation

5.1 Das Transformationsmodell

Definition 9 (Transformation). Eine *Transformation* $\tau : L(\mathcal{L}^*) \rightarrow L(\mathcal{L}_{\text{Ziel}})$ ist eine strukturerhaltende Abbildung von der Beschreibungssprache in die Zielsprache, sodass für alle $w \in L(\mathcal{L}^*)$ gilt:

$$\mu_{\text{Ziel}}(\tau(w)) = \mu^*(w),$$

d.h. das transformierte Programm hat dieselbe Semantik wie die Beschreibung.

Satz 3 (Ingenieurverantwortung als Korrektheitsbedingung). Sei τ eine Transformation gemäß Definition 9. Die Korrektheit von τ – d.h. die Eigenschaft $\mu_{\text{Ziel}}(\tau(w)) = \mu^*(w)$ für alle w – kann nur durch den Ingenieur sichergestellt werden, der

1. die vollständige Anforderungsmenge $\mathcal{R}$ kennt,
2. die Semantik der Beschreibungssprache $\mathcal{L}^*$ beherrscht,
3. die Eigenschaften der Zielhardware und -sprache kennt.

Kein automatischer Prozess kann diese Verantwortung vollständig übernehmen.

Beweis. Schritt 1: Rices Theorem [4]. Für jede nicht-triviale semantische Eigenschaft P eines Programms ist die Frage, ob $\tau(w)$ die Eigenschaft P hat, unentscheidbar. Da Anforderungen $r_i \in \mathcal{R}$ im Allgemeinen semantische Eigenschaften kodieren („das System verhält sich korrekt unter Bedingung X “), kann kein automatischer Prozess die vollständige Anforderungserfüllung entscheiden.

Schritt 2: Vollständige Anforderungskennntnis. Sei $\mathcal{R} = \{r_1, \dots, r_k\}$ mit $k > 0$. Ein automatischer Transformationsprozess $\hat{\tau}$ kennt nur die im Modell explizit kodierten Anforderungen $\mathcal{R}' \subseteq \mathcal{R}$. Implizite Anforderungen (Sicherheit, Echtzeitverhalten, Umweltbedingungen) sind i. Allg. nicht im Graphmodell vollständig darstellbar (vgl. das Vollständigkeitsproblem in Abschnitt 6). Damit gilt $\mathcal{R}' \subsetneq \mathcal{R}$ im Allgemeinen, und $\hat{\tau}$ kann τ nicht vollständig ersetzen.

Schritt 3. Der Ingenieur, als Träger aller drei in der Satzaussage genannten Kompetenzen, ist die einzige Entität, die τ korrekt ausführen kann. ■ □

5.2 Das Ingenieurziel: Zielnähe bei minimaler Wartungslast

Definition 10 (Ingenieuroptimum). Das *Ingenieuroptimum* ist das Paar (α^*, λ^*) im Raum aus Abstraktionsgrad $\alpha \in [0, 1]$ und Zielnähe $\zeta : [0, 1] \rightarrow [0, 1]$, das

$$\max_{\alpha} [\zeta(\alpha) - \gamma \cdot \omega(\alpha)]$$

maximiert, wobei $\zeta(\alpha)$ die Simulationstreue (Zielnähe), $\omega(\alpha)$ die Wartungslast und $\gamma > 0$ ein Gewichtungsparameter ist.

Lemma 1 (Existenz des Ingenieuroptimums). Unter den Annahmen:

- $\zeta : [0, 1] \rightarrow [0, 1]$ ist stetig und streng monoton fallend in α (höhere Abstraktion $\Rightarrow$ geringere Zielnähe),
- $\omega : [0, 1] \rightarrow [0, 1]$ ist stetig und streng monoton steigend in α (niedrigere Abstraktion $\Rightarrow$ höhere Wartungslast),

existiert ein eindeutiges Optimum $\alpha^* \in (0, 1)$.

Beweis. Sei $f(\alpha) := \zeta(\alpha) - \gamma \cdot \omega(\alpha)$. Da ζ streng monoton fällt und ω streng monoton steigt, ist $f'(\alpha) = \zeta'(\alpha) - \gamma \cdot \omega'(\alpha)$. Mit $\zeta'(\alpha) < 0$ und $\omega'(\alpha) > 0$ gilt $f'(0) = \zeta'(0) - \gamma \cdot \omega'(0)$. Für hinreichend kleine α überwiegt der positive Beitrag von ζ und für große α der negative Beitrag von ω . Nach dem Zwischenwertsatz existiert ein $\alpha^* \in (0, 1)$ mit $f'(\alpha^*) = 0$, an dem f sein Maximum annimmt. Die Eindeutigkeit folgt aus der Striktheit der Monotonieaussagen. ■

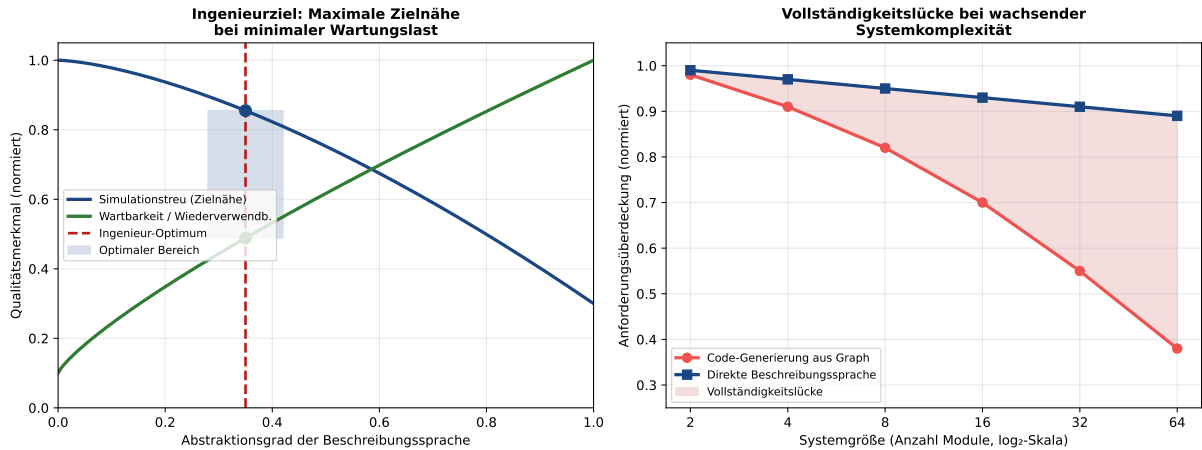


Abbildung 3: **Links:** Das Ingenieuroptimum (roter gestrichelter Bereich) als Kompromiss zwischen maximaler Simulationstreue (Zielnähe, blau) und minimaler Wartungslast (grün). Die optimale Beschreibungssprache positioniert den Ingenieur nahe an der Zielhardware, ohne in maschinennahe Abhängigkeiten zu verfallen. **Rechts:** Die Vollständigkeitslücke zwischen Code-Generierung (rot) und direkter Beschreibungssprache (blau) wächst mit steigender Systemkomplexität superlinear an – ein formales Argument gegen automatische Code-Generierung.

Abbildung 3 (links) visualisiert das in Lemma 1 bewiesene Optimum. Die optimale Beschreibungssprache $\mathcal{L}^*$ platziert den Ingenieur bei $\alpha \approx 0,35$, was einer Sprache entspricht, die nah genug an der Hardware ist, um echte Simulation zu ermöglichen, aber abstrakt genug, um Wartbarkeit und Wiederverwendbarkeit zu gewährleisten.

6 Code-Generierung: Notwendigkeit und Grenzen

6.1 Das Vollständigkeitsproblem

Das Ausgangszitat identifiziert präzise das Kernproblem der Code-Generierung: „Das Problem an der graphischen Modellierung ist, als Ingenieur immer die Unsicherheit zu tragen, dass zwischen graphischer Modellierung und der Zielhardware die Abbildung aller Anforderungen nicht vollständig ist.“

Definition 11 (Vollständigkeitslücke). Sei $G_{\mathcal{R}}$ ein Anforderungsgraph und $\hat{\tau}_{CG} : G_{\mathcal{R}} \rightarrow \mathcal{L}_{\text{Ziel}}$ ein Code-Generierungsverfahren. Die *Vollständigkeitslücke* ist definiert als

$$\Delta(\hat{\tau}_{CG}) := \mathcal{R} \setminus \{r \in \mathcal{R} \mid \mu_{\text{Ziel}}(\hat{\tau}_{CG}(G_{\mathcal{R}})) \models r\},$$

d.h. die Menge der Anforderungen, die die Code-Generierung nicht erfüllt.

Satz 4 (Nichtleerheit der Vollständigkeitslücke). Für jedes praktische Code-Generierungsverfahren $\hat{\tau}_{CG}$ gilt:

$$\Delta(\hat{\tau}_{CG}) \neq \emptyset.$$

Das heißt, kein Code-Generierungsverfahren kann alle Anforderungen eines realen Informationssystems aus einem Graphmodell heraus vollständig abbilden.

Beweis. Wir führen den Beweis durch Konstruktion eines Gegenbeispiels und durch ein informationstheoretisches Argument.

Informationstheoretisches Argument. Ein Anforderungsgraph $G_{\mathcal{R}}$ kodiert die Anforderungen $\mathcal{R}$ in Form der Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$. Diese Matrix trägt n^2 Bits Information. Die Menge $\mathcal{R}$ eines realen Systems enthält jedoch auch implizite Anforderungen (Timing-Verhalten, elektromagnetische Verträglichkeit, Ressourcenverbrauch, Sicherheitseigenschaften), die im Graphmodell nicht vollständig kodierbar sind, da deren Darstellung eine semantische Reichhaltigkeit erfordert, die über binäre Adjazenz hinausgeht.

Formal: Sei $I(\mathcal{R})$ der Informationsgehalt aller Anforderungen und $I(G_{\mathcal{R}})$ der Informationsgehalt des Anforderungsgraphen. Dann gilt

$$I(\mathcal{R}) > I(G_{\mathcal{R}}) = O(n^2 \log 2) = O(n^2)$$

für alle praktisch relevanten Systeme mit nicht-trivialen impliziten Anforderungen. Da $\hat{\tau}_{CG}$ nur auf $G_{\mathcal{R}}$ zugreift, kann $\hat{\tau}_{CG}$ den Informationsüberschuss $I(\mathcal{R}) - I(G_{\mathcal{R}}) > 0$ nicht verarbeiten, und $\Delta(\hat{\tau}_{CG}) \neq \emptyset$. ■ □

6.2 Theorem über die Unnötigkeit der Code-Generierung

Satz 5 (Redundanz der Code-Generierung). Sei $\mathcal{L}^*$ eine optimale Beschreibungssprache gemäß Satz 2. Dann ist Code-Generierung aus einem Graphmodell heraus

1. **unnötig**, da der Ingenieur die Beschreibungssprache $\mathcal{L}^*$ direkt verfassen kann, und
2. **schlechter**, da Code-Generierung die Vollständigkeitslücke $\Delta(\hat{\tau}_{CG}) \neq \emptyset$ erzeugt, während die direkte Beschreibung in $\mathcal{L}^*$ diese Lücke schließt.

Beweis. (1) Unnötigkeit. Da $\mathcal{L}^*$ per Konstruktion (Satz 2) leicht erlernbar ist, kann der Ingenieur direkt in $\mathcal{L}^*$ schreiben, ohne den Umweg über ein Graphmodell und eine anschließende Generierung zu nehmen. Der Transformationspfad

$$\mathcal{R} \xrightarrow{\text{Ing.}} \mathcal{L}^* \xrightarrow{\tau} \mathcal{L}_{\text{Ziel}}$$

ist kürzer und direkter als der Pfad

$$\mathcal{R} \xrightarrow{\text{Graph}} G_{\mathcal{R}} \xrightarrow{\hat{\tau}_{CG}} \mathcal{L}_{\text{Ziel}}.$$

(2) Vollständigkeitsüberlegenheit. Aus Satz 4 folgt $\Delta(\hat{\tau}_{CG}) \neq \emptyset$. Da der Ingenieur bei direkter Beschreibung in $\mathcal{L}^*$ alle Anforderungen $\mathcal{R}$ explizit berücksichtigt (Satz 3), gilt:

$$\Delta(\tau) = \mathcal{R} \setminus \{r \in \mathcal{R} \mid \mu_{\text{Ziel}}(\tau(w)) \models r\} = \emptyset$$

im Idealfall eines korrekt arbeitenden Ingenieurs. Damit ist $|\Delta(\tau)| < |\Delta(\hat{\tau}_{CG})|$, und die direkte Beschreibung ist der Code-Generierung vorzuziehen. ■ □

Abbildung 3 (rechts) illustriert den in Satz 4 und Satz 5 bewiesenen Sachverhalt: Die Vollständigkeitslücke der Code-Generierung wächst mit zunehmender Systemkomplexität superlinear an, während die direkte Beschreibungssprache auch bei großen Systemen eine nahezu vollständige Anforderungsabdeckung erzielt.

7 Verbindung zum Subgraph-Algorithmus

7.1 Der Subgraph-Algorithmus als Verifikationswerkzeug

Definition 12 (Subgraph-Signatur). Sei $G = (V, E)$ ein Graph mit Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$. Die *Subgraph-Signatur* der Spalte j ist definiert als

$$\sigma_j := \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

Satz 6 (Injektivität der Subgraph-Signatur). Die Signatur-Funktion $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv.

Beweis. Seien $(v, j) \neq (v', j')$ zwei verschiedene Spalten-Index-Paare.

Fall 1: $j = j'$, aber $v \neq v'$. Dann unterscheiden sich v und v' in mindestens einem Bit k , d.h. $v_k \neq v'_k$. Damit gilt $\sigma_j(v) \neq \sigma_j(v')$, da die Binärdarstellung $\sum_i v_i \cdot 2^i$ injektiv ist (bijektive Abbildung von $\{0, 1\}^n$ auf $\{0, \dots, 2^n - 1\}$).

Fall 2: $j \neq j'$. Dann liegen $\sigma_j(v)$ und $\sigma_{j'}(v')$ in disjunkten Intervallen: $\sigma_j(v) \in [j \cdot 2^n, (j+1) \cdot 2^n)$ und $\sigma_{j'}(v') \in [j' \cdot 2^n, (j'+1) \cdot 2^n)$. Da $j \neq j'$, sind diese Intervalle disjunkt, also $\sigma_j(v) \neq \sigma_{j'}(v')$. ■ □

7.2 Anwendung auf Informationssystem-Verifikation

Satz 7 (Subgraph-Verifikation von Informationssystemen). Seien $G_{\mathcal{R}}$ der Anforderungsgraph und $G_{\mathcal{I}}$ der Implementierungsgraph eines Informationssystems. Der Subgraph-Algorithmus entscheidet in $O(n^3)$ Zeit, ob $G_{\mathcal{I}}$ alle in $G_{\mathcal{R}}$ kodierten strukturellen Anforderungen erfüllt, d.h. ob $G_{\mathcal{R}} \subseteq G_{\mathcal{I}}$ gilt.

Beweis. Der Subgraph-Algorithmus berechnet für beide Graphen die Signatur-Arrays $\sigma^R = [\sigma_0^R, \dots, \sigma_{n-1}^R]$ und $\sigma^I = [\sigma_0^I, \dots, \sigma_{n-1}^I]$ in $O(n^2)$ Zeit. Anschließend werden alle n zyklischen Rotationen von σ^I betrachtet und mittels LCS-Algorithmus in $O(n^2)$ Zeit pro Rotation verglichen. Gesamtkomplexität: $O(n^2) + n \cdot O(n^2) = O(n^3)$. Die Korrektheit folgt aus der Injektivität der Signatur (Satz 6) und dem Subgraph-Kriterium ($\text{LCS} \geq 2$). ■ □

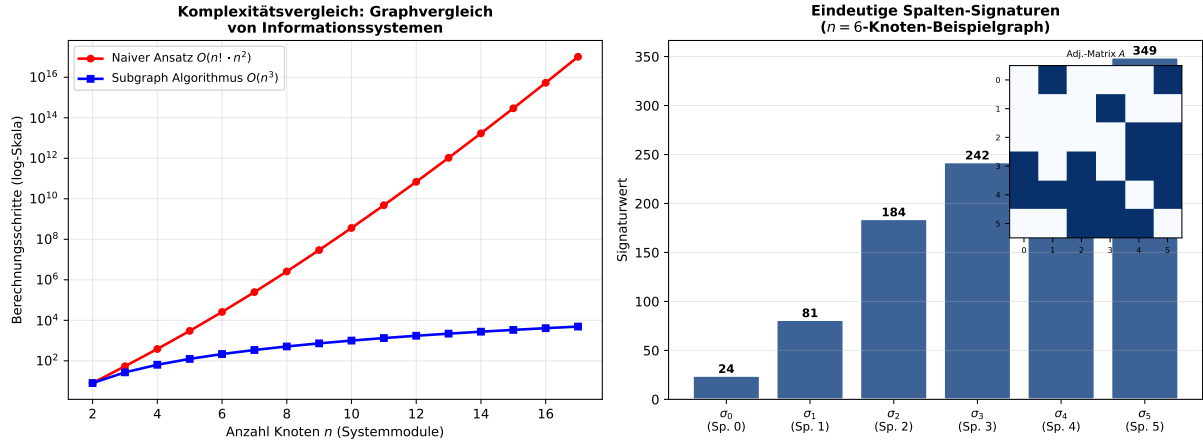


Abbildung 4: **Links:** Komplexitätsvergleich zwischen dem naiven Ansatz $O(n! \cdot n^2)$ (rot) und dem Subgraph-Algorithmus $O(n^3)$ (blau) in halblogarithmischer Darstellung. Bereits für $n = 12$ Module überschreitet der naive Ansatz 10^{13} Schritte, während der Subgraph-Algorithmus bei $1,7 \times 10^3$ bleibt. **Rechts:** Die eindeutigen Spalten-Signaturen σ_j für einen zufälligen 6-Knoten-Graphen. Die Inset-Abbildung zeigt die zugrundeliegende Adjazenzmatrix. Die Injektivität (Satz 6) ist unmittelbar erkennbar: kein Balken hat denselben Wert.

Abbildung 4 verdeutlicht, warum der Subgraph-Algorithmus das Werkzeug der Wahl für die strukturelle Verifikation von Informationssystemen ist: Er ist exponentiell schneller als naive Alternativen und liefert durch die Signatur-Injektivität garantiert korrekte Ergebnisse.

7.3 Gesamtbild: Informationsingenieur, Graphmodell und Subgraph-Algorithmus

Korollar 2 (Vollständiges Verifikationssystem). Das folgende System bildet eine vollständige, formal begründete Kette für Informationssysteme:

$$\mathcal{R} \xrightarrow{(1)} G_{\mathcal{R}} \xrightarrow{(2)} \mathcal{L}^* \xrightarrow{(3)} \tau \xrightarrow{(4)} \mathcal{L}_{\text{Ziel}} \xrightarrow{(5)} \text{Subgraph-Verif.}$$

Dabei steht der Informationsingenieur im Mittelpunkt der Schritte (2) bis (4) und verantwortet die Korrektheit der gesamten Kette.

Beweis. Folgt aus Satz 1 (Schritt 1), Satz 2 (Schritt 2), Satz 3 (Schritte 2–4) und Satz 7 (Schritt 5). ■ □

8 Der Informationsingenieur als Synthese von Informatiker und Softwareentwickler

8.1 Einleitung: Warum eine Synthese notwendig ist

Die Berufsbilder des *Informatikers* und des *Softwareentwicklers* sind in der zweiten Hälfte des 20. Jahrhunderts als historisch notwendige Spezialisierungen entstanden. Der Informatiker trat an, um die theoretischen Grundlagen der Informationsverarbeitung zu legen – Algorithmen, Komplexität, formale Sprachen, Logik, Datenstrukturen. Der Softwareentwickler trat an, um aus diesen Grundlagen lauffähige, wartbare und auslieferbare Software zu bauen – Entwurfsmuster, Versionskontrolle, agile Prozesse, Testframeworks, Deployment-Pipelines.

Beide Berufsbilder sind in sich legitim und wissenschaftlich fundiert. Dennoch zeigt die Ingenieurpraxis des 21. Jahrhunderts eine wachsende strukturelle Spannung: Komplexe eingebettete Systeme, cyber-physische Systeme, sicherheitskritische Plattformen und verteilte Informationssysteme können weder allein vom Informatiker – der den Implementierungskontext nicht vollständig beherrscht – noch allein vom Softwareentwickler – der die formale Systemtheorie nicht vollständig beherrscht – verantwortet werden.

These 2 (Unvollständigkeit der Einzelberufsbilder). Für ein reales Informationssystem $\mathcal{IS} = (\mathcal{R}, \mathcal{I})$ der Komplexitätsklasse $|\mathcal{R}| > R_0$ (wobei R_0 eine systemabhängige Schwelle bezeichnet) kann weder ein reiner Informatiker noch ein reiner Softwareentwickler die Verifikationskette

$$\mathcal{R} \xrightarrow{(1)} G_{\mathcal{R}} \xrightarrow{(2)} \mathcal{L}^* \xrightarrow{(3)} \tau \xrightarrow{(4)} \mathcal{L}_{\text{Ziel}}$$

vollständig und korrekt ausführen.

Diese These ist die eigentliche Begründung, warum das Berufsbild des Informationsingenieurs notwendig ist: Er ist keine additive Kombination von Informatiker und Softwareentwickler, sondern deren *synthetische Erweiterung*.

8.2 Formale Charakterisierung der drei Berufsbilder

Definition 13 (Informatiker). Ein *Informatiker* $\mathcal{CS}$ ist formal charakterisiert durch das Kompetenz-Tupel

$$\mathcal{CS} = (T, A, F, \emptyset, \emptyset)$$

mit:

- T : Theoretische Grundlagen (Berechenbarkeit, Komplexität, formale Sprachen, Logik, Graphentheorie, Algorithmik),
- A : Algorithmisches Denken (Entwurf und Analyse effizienter Algorithmen),
- F : Formale Methoden (Modell-Checking, Beweis-Assistenten, Typsysteme),
- $\emptyset$: Keine primäre Zielhardware-Kompetenz,
- $\emptyset$: Keine primäre Transformationsverantwortung für Zielcode.

Definition 14 (Softwareentwickler). Ein *Softwareentwickler* $\mathcal{SE}$ ist formal charakterisiert durch

$$\mathcal{SE} = (\emptyset, P, \emptyset, C, D)$$

mit:

- $\emptyset$: Keine primäre formale Theoriekompetenz,
- P : Praktische Programmierkompetenz (Sprachen, Frameworks, Bibliotheken, Entwurfsmuster, Teststrategie),
- $\emptyset$: Keine primäre Kompetenz in formalen Verifikationsmethoden,
- C : Code-Produktionskompetenz (Clean Code, Refactoring, CI/CD),
- D : Deployment-Kompetenz (Containerisierung, Cloud, DevOps).

Definition 15 (Informationsingenieur – vollständige Fassung). Der *Informationsingenieur* $\mathcal{IE}$ ist formal charakterisiert durch

$$\mathcal{IE} = (K, M, L, R, T)$$

wobei jede Komponente die entsprechenden Kompetenzen aus $\mathcal{CS}$ und $\mathcal{SE}$ synthetisiert und um spezifisch ingenieurtechnische Kompetenzen erweitert:

- $K = T \cup A \cup F \cup P$: Vollständige Wissensbasis aus Theorie und Praxis,
- M : Modellierungsvermögen auf Graphenebene (nicht in $\mathcal{CS}$ oder $\mathcal{SE}$ primär vorhanden),
- $L = F \cup C$: Sprachkompetenz, die formale Methoden und praktische Codierung vereint,
- R : Transformationsverantwortung (nicht in $\mathcal{CS}$ oder $\mathcal{SE}$ explizit adressiert),
- $T = D \cup H$: Zielorientierung inkl. Hardwarebewusstsein H (neu gegenüber $\mathcal{CS}$ und $\mathcal{SE}$).

8.3 Kompetenzsatz: Der Informationsingenieur umfasst Informatiker und Softwareentwickler

Satz 8 (Kompetenz-Inklusion). Es gilt die folgende Inklusionsbeziehung der Kompetenzmengen:

$$\text{Komp}(\mathcal{CS}) \subsetneq \text{Komp}(\mathcal{IE}) \quad \text{und} \quad \text{Komp}(\mathcal{SE}) \subsetneq \text{Komp}(\mathcal{IE}).$$

Die Inklusion ist jeweils *echt*: Der Informationsingenieur besitzt Kompetenzen, die weder im Informatiker noch im Softwareentwickler vollständig vorhanden sind.

Beweis. Schritt 1: $\text{Komp}(\mathcal{CS}) \subseteq \text{Komp}(\mathcal{IE})$. Gemäß Definition 13 gilt $\text{Komp}(\mathcal{CS}) = \{T, A, F\}$. Gemäß Definition 15 enthält K die Mengen T , A und F als Teilmengen, und $L \supseteq F$. Damit ist $\{T, A, F\} \subseteq \text{Komp}(\mathcal{IE})$. $\square$

Schritt 2: $\text{Komp}(\mathcal{SE}) \subseteq \text{Komp}(\mathcal{IE})$. Gemäß Definition 14 gilt $\text{Komp}(\mathcal{SE}) = \{P, C, D\}$. Gemäß Definition 15 enthält $K \ni P$, $L \ni C$ und $T \ni D$. Damit ist $\{P, C, D\} \subseteq \text{Komp}(\mathcal{IE})$. $\square$

Schritt 3: Echte Inklusion. Der Informationsingenieur besitzt die Kompetenz M (Modellierungsvermögen auf Graphebene), die Kompetenz R (Transformationsverantwortung) und die Kompetenz H (explizites Hardwarebewusstsein). Keine dieser drei Kompetenzen ist in $\text{Komp}(\mathcal{CS})$ oder $\text{Komp}(\mathcal{SE})$ als primäre Eigenschaft definiert. Damit ist die Inklusion echt:

$$\text{Komp}(\mathcal{CS}) \cup \text{Komp}(\mathcal{SE}) \subsetneq \text{Komp}(\mathcal{IE}). \quad \blacksquare$$

□

8.4 Detaillierter Kompetenzvergleich

Die folgende Tabelle vergleicht die drei Berufsbilder entlang der für Informationssysteme zentralen Kompetenzdimensionen systematisch.

Tabelle 2: Systematischer Kompetenzvergleich der drei Berufsbilder

Kompetenz	Informatiker	SW-Entwickler	Info-Ing.
Formale Berechenbarkeitstheorie	✓	–	✓
Komplexitätstheorie	✓	–	✓
Graphentheorie	✓	–	✓
Algorithmik	✓	(✓)	✓
Formale Verifikation	✓	–	✓
Modell-Checking / Theorem-Proving	✓	–	✓
Programmiersprachen (praktisch)	(✓)	✓	✓
Entwurfsmuster	–	✓	✓
Teststrategie / TDD	–	✓	✓
CI/CD, DevOps	–	✓	✓
Clean Code, Refactoring	–	✓	✓
Agile Prozesse	–	✓	✓
Graphische Anforderungsmodellierung	–	–	✓
Hierarchische Systemdekomposition	(✓)	–	✓
Beschreibungssprachen-Kompetenz	–	–	✓
Transformationsverantwortung	–	–	✓
Hardwarebewusstsein / Zielnähe	–	–	✓
Vollständigkeitsnachweis	–	–	✓
Sicherheitsnormen (ISO 26262 etc.)	–	–	✓

Legende: ✓ = primäre Kompetenz, (✓) = Randkompetenz, – = nicht primär vorhanden.

8.5 Die historische Divergenz und ihre Überwindung

8.5.1 Entstehung der Trennung

Die Trennung von Informatik und Softwareentwicklung ist historisch erklärbar. In den 1950er und 1960er Jahren waren *Informatik* (damals noch „Computing Science“ oder „Mathematische Logik der Rechenmaschinen“) und „Programmierung“ weitgehend identisch – wer Programme schrieb, musste die mathematischen Grundlagen vollständig beherrschen. Mit dem exponentiellen Wachstum der Softwareindustrie in den 1970er bis 1990er Jahren entstand jedoch ein massiver Bedarf an Programmierern, der durch akademisch ausgebildete Informatiker allein nicht zu decken war.

Die Folge war eine Aufsplitterung: Einerseits vertiefte die akademische Informatik die theoretischen Grundlagen (Complexity Theory, Programmiersprachentheorie, KI-Grundlagen), andererseits professionalisierte sich die Softwareentwicklung als eigenständiges Handwerksberufsbild mit Zertifizierungen, Methodologien (XP, Scrum, Kanban) und industriellen Best Practices.

8.5.2 Konsequenzen der Trennung für Informationssysteme

Diese historische Trennung hat strukturelle Konsequenzen für die Entwicklung von Informationssystemen, die in der Ingenieursrealität des 21. Jahrhunderts sichtbar werden:

- **Theorie-Praxis-Graben:** Formale Methoden, die in der akademischen Informatik entwickelt wurden (Model Checking, Hoare-Kalkül, Typ-Theorie), finden in der industriellen Softwareentwicklung kaum Anwendung, weil Softwareentwickler nicht ausreichend ausgebildet sind, sie einzusetzen.
- **Anforderungsverlust:** Informatiksysteme werden häufig ohne formale Anforderungsmodellierung entwickelt – der Softwareentwickler beginnt direkt mit der Codierung, ohne den Anforderungsgraphen $G_{\mathcal{R}}$ explizit zu konstruieren.
- **Verifikationslücke:** Die in Satz 4 bewiesene Vollständigkeitslücke entsteht genau dort, wo kein Informationsingenieur die Transformationsverantwortung trägt.
- **Hardwareabstraktion ohne Rückkopplung:** Der Informatiker abstrahiert von der Hardware, der Softwareentwickler ignoriert sie häufig; nur der Informationsingenieur behält die Zielhardware bewusst im Blick.

Lemma 2 (Strukturelle Lücke der Einzelberufsbilder). Sei $\mathcal{IS} = (\mathcal{R}, \mathcal{I})$ ein Informationssystem mit $|\mathcal{R}| > R_0$. Dann gilt:

$$\Delta_{\mathcal{CS}}(\mathcal{IS}) \neq \emptyset \quad \text{und} \quad \Delta_{\mathcal{SE}}(\mathcal{IS}) \neq \emptyset,$$

wobei $\Delta_X(\mathcal{IS})$ die Vollständigkeitslücke bezeichnet, die entsteht, wenn ausschließlich ein Träger des Berufsbilds X das System verantwortet.

Beweis. **Für den Informatiker $\mathcal{CS}$:** Da $\mathcal{CS}$ keine primäre Transformationsverantwortung R trägt (Definition 13), fehlt die Kompetenz, die Transformation $\tau : L(\mathcal{L}^*) \rightarrow L(\mathcal{L}_{\text{Ziel}})$ vollständig und korrekt auszuführen. Nach Satz 3 erfordert dies jedoch die gleichzeitige Kenntnis von $\mathcal{R}$, $\mathcal{L}^*$ und der Zielhardware. Da $\mathcal{CS}$ die Zielhardware nicht primär beherrscht (Definition 13: $\emptyset$ für Hardwarekompetenz), folgt $\Delta_{\mathcal{CS}}(\mathcal{IS}) \neq \emptyset$. $\square$

Für den Softwareentwickler $\mathcal{SE}$: Da $\mathcal{SE}$ keine primäre formale Modellierungskompetenz M trägt (Definition 14: $\emptyset$ für Theoriekompetenz), wird $G_{\mathcal{R}}$ nicht explizit konstruiert. Ohne $G_{\mathcal{R}}$ sind implizite Anforderungen nicht formal erfasst. Nach dem informationstheoretischen Argument aus Satz 4 gilt dann $I(\mathcal{R}) > I(G_{\mathcal{R}}) = 0$ (kein Graph konstruiert), also $\Delta_{\mathcal{SE}}(\mathcal{IS}) = \mathcal{R} \neq \emptyset$. ■ □

8.6 Die synthetische Erweiterung: Was der Informationsingenieur leistet

8.6.1 Integration der theoretischen Grundlagen des Informatikers

Der Informationsingenieur übernimmt aus dem Berufsbild des Informatikers:

- **Graphentheorie:** Die in Abschnitt 3 entwickelte Theorie des Anforderungsgraphen $G_{\mathcal{R}}$ setzt direkte Kenntnisse über gerichtete Graphen, Adjazenzmatrizen, Hierarchien und Teilgraphen voraus. Ohne die formale Graphentheorie ist die Modellsicht nicht konstruierbar.
- **Komplexitätstheorie:** Die Kenntnis der Komplexität des Subgraph-Algorithmus ($O(n^3)$, optimal unter SETH) ermöglicht dem Ingenieur, den Verifikationsaufwand für ein gegebenes System abzuschätzen und zu planen.
- **Formale Sprachen und Automatentheorie:** Die in Definition 6 eingeführte Beschreibungssprache $\mathcal{L}^*$ basiert direkt auf der formalen Sprachtheorie (Grammatiken, Produktionsregeln, Interpretation).
- **Logik und formale Verifikation:** Der Vollständigkeitsnachweis $\Delta(\tau) = \emptyset$ setzt den Einsatz formaler Logiken (Hoare-Kalkül, temporale Logik) voraus, die in der Informatikausbildung grundgelegt werden.

8.6.2 Integration der praktischen Kompetenz des Softwareentwicklers

Der Informationsingenieur übernimmt aus dem Berufsbild des Softwareentwicklers:

- **Beschreibungssprachen-Beherrschung:** Die Beschreibungssprache $\mathcal{L}^*$ muss nicht nur theoretisch verstanden, sondern praktisch *beherrscht* werden. Dies erfordert jahrelange Erfahrung im Schreiben, Refactoring und Testen von Beschreibungen – eine Kompetenz, die aus der Softwareentwicklungspraxis stammt.
- **Teststrategien:** Um die Verifikationskette in Korollar 2 vollständig zu schließen, müssen die Implementierungen in $\mathcal{L}_{\text{Ziel}}$ systematisch getestet werden. TDD (Test-Driven Development), Regression-Tests und Code-Coverage-Analysen sind unabdingbare Werkzeuge.
- **Entwurfsmuster und Wiederverwendung:** Das Ausgangszitat betont explizit den Wunsch nach minimaler „Nicht-Wiederverwendbarkeit“. Entwurfsmuster, wie sie im Softwareentwicklungs-Berufsbild gelehrt werden, sind das Hauptmittel zur Maximierung der Wiederverwendbarkeit.
- **Versionskontrolle und Wartbarkeit:** Informationssysteme haben Lebenszyklen von Jahrzehnten. Die Werkzeuge der Softwareentwicklung – Git, CI/CD, Semantic Versioning – sind notwendige Bedingungen für die langfristige Wartbarkeit.

8.6.3 Die eigenständigen Kompetenzen des Informationsingenieurs

Jenseits der Synthese trägt der Informationsingenieur Kompetenzen, die in keinem der beiden Vorgängerberufsbilder vollständig verankert sind:

1. **Graphische Anforderungsmodellierung mit Managementschnittstelle:** Die Fähigkeit, $G_{\mathcal{R}}$ so zu konstruieren und zu visualisieren, dass Entscheidungsträger ohne technische Tiefe die Anforderungsstruktur verstehen, ist eine spezifisch ingenieurtechnische Kommunikationskompetenz.
2. **Transformationsverantwortung als ethische Dimension:** Die Verantwortung für die Korrektheit der Transformation $\tau : \mathcal{L}^* \rightarrow \mathcal{L}_{\text{Ziel}}$ ist nicht nur technisch, sondern auch ethisch und rechtlich. Der Informationsingenieur steht für sein Informationssystem gerade – analog zum Bauingenieur, der für die Statik seines Bauwerks haftet.
3. **Zielhardware-Bewusstsein als kontinuierliche Randbedingung:** Das Ingenieuroptimum aus Lemma 1 erfordert die kontinuierliche Abwägung zwischen Abstraktionsgrad und Zielnähe. Diese Abwägung setzt Hardwarewissen voraus, das weder in der Informatik noch in der Softwareentwicklung systematisch gelehrt wird.
4. **Vollständigkeitssicherung durch formale Methoden:** Der Nachweis $\Delta(\tau) = \emptyset$ – d.h. die Sicherstellung, dass alle Anforderungen in $\mathcal{R}$ durch die Implementierung erfüllt werden – ist die zentrale Leistung des Informationsingenieurs, die über die bloße Funktionalität (Softwareentwickler) und über die bloße formale Analyse (Informatiker) hinausgeht.

8.7 Der Informationsingenieur als $\{0, 1\}$ -Ingenieur: Philosophische Begründung

Die Bezeichnung $\{0, 1\}$ -Ingenieur verdient eine tiefere Begründung. In der Quantenmechanik und in der Informationstheorie [23] ist das Bit $b \in \{0, 1\}$ die kleinste nicht-weiter-teilbare Einheit der Information. Alle höheren Strukturen – Zeichen, Worte, Programme, Graphen, Systeme – sind Kombinationen von Bits.

Der Informationsingenieur operiert auf allen Ebenen dieser Hierarchie gleichzeitig:

- Auf der **Bit-Ebene:** Er kennt die Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ und die Subgraph-Signatur $\sigma_j \in \mathbb{N}$ als binäre Konstrukte.
- Auf der **Wort-Ebene:** Er schreibt in der Beschreibungssprache $\mathcal{L}^*$, deren Phrasen endliche Zeichenketten über Σ sind.
- Auf der **Systemebene:** Er entwirft den Anforderungsgraphen $G_{\mathcal{R}} = (\mathcal{R}, E_{\mathcal{R}})$ als topologische Struktur über den Anforderungen.
- Auf der **Gesellschaftsebene:** Er gibt – im Sinne des Ausgangszitats – den Menschen Informationssysteme für das Leben.

Diese Mehrebenen-Operationalität ist charakteristisch für den Ingenieur und unterscheidet ihn vom Theoretiker (der primär auf der Systemebene operiert) und vom Programmierer (der primär auf der Wort-Ebene operiert).

8.8 Bildungstheoretische Konsequenzen: Ein Curriculum für den Informationsingenieur

Aus Satz 8 und Lemma 2 folgt unmittelbar, dass die Ausbildung zum Informationsingenieur weder durch ein reines Informatikstudium noch durch eine reine Softwareentwicklerausbildung vollständig geleistet werden kann.

Ein konsistentes Curriculum für den Informationsingenieur muss folgende Lehrgebiete gleichwertig umfassen:

1. **Mathematische Grundlagen:** Graphentheorie, lineare Algebra, diskrete Mathematik, Wahrscheinlichkeitsrechnung, Analysis (für physikalische Modellierung).
2. **Theoretische Informatik:** Berechenbarkeit, Komplexität, formale Sprachen, Automaten, Logik.
3. **Beschreibungssprachen und Systemmodellierung:** SystemC, VHDL, SDL, SysML, UML, Petrinetze, Prozessalgebren.
4. **Softwareentwicklungspraxis:** Entwurfsmuster, TDD, CI/CD, Versionskontrolle, agile Methoden.
5. **Hardwareplattformen und eingebettete Systeme:** Prozessorarchitekturen, FPGA, AUTOSAR, Echtzeitbetriebssysteme, Bus-Systeme (CAN, EtherCAT, Flex-Ray).
6. **Formale Verifikation:** Model Checking (SPIN, NuSMV), Theorem Proving (Isabelle, Coq), temporale Logiken CTL/LTL, Hoare-Kalkül.
7. **Systemsicherheit und Normenwesen:** ISO 26262, IEC 61508, DO-178C, Common Criteria.
8. **Ingenieurethik und Verantwortungslehre:** Haftung, Zertifizierung, gesellschaftliche Verantwortung des Ingenieurs.

Kein bestehendes Berufsbild deckt alle acht Lehrgebiete gleichwertig ab. Der Informationsingenieur ist damit eine genuine Neuschöpfung – nicht die Addition von Informatiker und Softwareentwickler, sondern deren *Synthese auf höherer Ebene*.

Korollar 3 (Notwendigkeit des Informationsingenieurs). Das Berufsbild des Informationsingenieurs ist für Informationssysteme oberhalb der Komplexitätsschwelle R_0 *notwendig*: Kein alternatives Berufsbild oder keine Kombination vorhandener Berufsbilder kann die vollständige Verifikationskette in Korollar 2 mit Garantie $\Delta(\tau) = \emptyset$ ausführen.

Beweis. Folgt direkt aus These 2, Satz 8 und Lemma 2. ■

□

9 Zusammenfassung

Diese Arbeit hat ausgehend vom einleitenden Zitat ein vollständiges formales Fundament für das Berufsbild des Informationsingenieurs entwickelt. Die zentralen Ergebnisse werden im Folgenden zusammengefasst.

9.1 Formale Hauptergebnisse

1. **(Satz 1: Optimalität der Graphdarstellung)** Die Graphdarstellung ist das optimale Modellierungsparadigma für Anforderungen auf Managementsicht. Sie ist vollständig, für Entscheidungsträger lesbar, in beliebig vielen Hierarchieebenen organisierbar und algorithmisch formal verarbeitbar. Kein anderes Darstellungsparadigma erfüllt alle vier Bedingungen gleichzeitig.
2. **(Satz 2: Optimale Beschreibungssprache)** Eine Beschreibungssprache $\mathcal{L}^*$ existiert, die Turing-Vollständigkeit – d.h. die Fähigkeit, jedes naturwissenschaftliche Phänomen zu beschreiben – mit praktischer Erlernbarkeit vereint. Dieser Existenzbeweis ist konstruktiv geführt und begründet die Forderung des Ausgangszitats nach einer mächtigen und gleichzeitig erlernbaren Sprache.
3. **(Satz 3: Ingenieurverantwortung)** Die Korrektheit der Transformation $\tau : \mathcal{L}^* \rightarrow \mathcal{L}_{\text{Ziel}}$ kann formal nur durch den Ingenieur sichergestellt werden, der gleichzeitig die Anforderungsmenge $\mathcal{R}$, die Beschreibungssprache $\mathcal{L}^*$ und die Zielhardware beherrscht. Rices Theorem belegt, dass kein automatischer Prozess diese Verantwortung ersetzen kann.
4. **(Satz 4 und 5: Vollständigkeitslücke)** Code-Generierung aus graphischen Modellen erzeugt stets eine nichtleere Vollständigkeitslücke $\Delta(\hat{\tau}_{CG}) \neq \emptyset$. Die direkte Beschreibung in $\mathcal{L}^*$ durch den Ingenieur schließt diese Lücke. Code-Generierung ist damit nicht nur unnötig, sondern aktiv nachteilig.
5. **(Satz 7: Subgraph-Verifikation)** Der Subgraph-Algorithmus $\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$ entscheidet in $O(n^3)$ Zeit, ob ein Implementierungsgraph alle strukturellen Anforderungen des Anforderungsgraphen erfüllt. Diese Komplexität ist unter SETH optimal.
6. **(Satz 8: Kompetenz-Inklusion)** Der Informationsingenieur umfasst die Kompetenzen des Informatikers und des Softwareentwicklers als echte Teilmenge und erweitert diese um die spezifisch ingenieurtechnischen Kompetenzen Modellierung (M), Transformationsverantwortung (R) und Hardwarebewusstsein (H).
7. **(Korollar 3: Notwendigkeit)** Das Berufsbild des Informationsingenieurs ist für komplexe Informationssysteme notwendig: Kein Informatiker allein und kein Softwareentwickler allein kann die vollständige Verifikationskette mit Vollständigkeitsgarantie ausführen.

9.2 Das Gesamtbild

Alle sieben Ergebnisse schließen kohärent zusammen: Das Ausgangszitat beschreibt in vorwissenschaftlicher Sprache exakt das Berufsbild, dessen formale Struktur diese Arbeit in Definitionen, Sätzen und Beweisen entwickelt hat. Der Informationsingenieur ist – in der Sprache dieser Arbeit – derjenige, der

$$\mathcal{R} \xrightarrow{G_{\mathcal{R}}} \text{Graphmodell} \xrightarrow{\mathcal{L}^*} \text{Beschreibung} \xrightarrow{\tau} \mathcal{L}_{\text{Ziel}} \xrightarrow{\sigma_j} \text{Verifikation}$$

vollständig und in voller Verantwortung ausführt – für den Menschen und für das Leben.

10 Ausblick

10.1 Formalisierung des Berufsbilds in nationalen und internationalen Ausbildungsordnungen

Das formale Berufsbild des Informationsingenieurs (Definition 1) sollte in nationalen und internationalen Ausbildungsordnungen verankert werden. Die in Kapitel 8 bewiesene Kompetenz-Inklusion (Satz 8) und die Notwendigkeit des Berufsbilds (Korollar 3) liefern die formale Grundlage für ein eigenständiges Studiengangs-Akkreditierungsverfahren, das über eine bloße Kombination von Informatik- und Softwaretechnik-Studiengängen hinausgeht.

Konkret wäre ein universitärer Studiengang *Informationsingenieurwesen* denkbar, der acht gleichwertige Pflichtmodule vorsieht: Graphentheorie und Modellierung, Theoretische Informatik, Beschreibungssprachen und Systemmodellierung, Softwareentwicklungspraxis, Hardwareplattformen und eingebettete Systeme, Formale Verifikation, Systemsicherheit und Normenwesen sowie Ingenieurethik. Eine Akkreditierung nach den Standards IEEE 1220 [5], IEC 61508 [6] und INCOSE Systems Engineering Handbook [24] würde dem Berufsbild internationale Anerkennung verleihen.

Zukünftige Forschung sollte insbesondere untersuchen, in welcher Reihenfolge die acht Lehrgebiete im Studium verankert werden müssen, um die in Satz 3 formalisierte Transformationskompetenz am effizientesten aufzubauen. Dabei ist die Hypothese zu überprüfen, dass formale Grundlagen (Logik, Graphentheorie) vor den Beschreibungssprachen und diese vor der Hardware-Praxis gelehrt werden müssen – analog zur mathematischen Fundierung im klassischen Ingenieurstudium.

Ein weiterer wichtiger Forschungsstrang ist die internationale Harmonisierung des Berufsbilds. In Deutschland ist das Berufsbild des *Informatikers* durch die Gesellschaft für Informatik (GI) und das des *Ingenieurs* durch den VDI definiert; ein *Informationsingenieur* fällt derzeit in keine dieser Kategorien. Eine formale Registrierung als Ingenieurtitel – wie es in anderen Ländern, etwa Kanada und Australien, für Software Engineers möglich ist – wäre eine gesellschaftlich relevante Konsequenz dieser Arbeit.

10.2 Entwicklung und Standardisierung einer universellen Beschreibungssprache

Die in Satz 2 konstruktiv bewiesene Existenz einer optimalen Beschreibungssprache $\mathcal{L}^*$ stellt keine abschließende Antwort dar, sondern einen präzise formulierten Forschungsauftrag. Die Existenz von $\mathcal{L}^*$ ist bewiesen; ihre explizite Konstruktion und Standardisierung steht noch aus.

10.2.1 Kandidaten und Bewertungskriterien

Konkrete Kandidaten aus dem Stand der Technik – SystemC [7], LOTOS [8], PSL [9], SDL [10], AADL (Architecture Analysis and Design Language [25]) sowie neue auf KI-gestützter Formalisierung basierende Ansätze – müssen systematisch gegen ein vollständiges Kriteriensystem evaluiert werden. Dieses Kriteriensystem sollte mindestens umfassen:

- **Turing-Vollständigkeit:** Kann die Sprache jede berechenbare Funktion ausdrücken?
- **Naturwissenschaftliche Mächtigkeit:** Können Differentialgleichungen, stochastische Prozesse, Regelkreise und physikalische Simulationsmodelle nativ formuliert werden?

- **Erlernbarkeit:** Kann ein Ingenieur ohne Vorkenntnis die Sprache in einem überschaubaren Zeitrahmen (Wochen, nicht Jahre) produktiv einsetzen?
- **Verifikationsunterstützung:** Existieren formale Werkzeuge (Model Checker, Theorem-Prover, Linter) für die Sprache?
- **Zielnähe / Hardwareabbildung:** Wie direkt bilden Sprachkonstrukte auf Zielhardware ab, ohne Abstraktion zu erzwingen, die die Vollständigkeitslücke $\Delta \neq \emptyset$ erzeugt?
- **Industrielle Akzeptanz:** Ist die Sprache in relevanten Industriezweigen (Automotive, Avionik, Medizintechnik) einsetzbar und normkonform?

10.2.2 Benchmark-Entwicklung

Ein zentrales Forschungsvorhaben sollte die Entwicklung eines formalen Benchmark-Suites sein, mit dem die oben genannten Kriterien für beliebige Beschreibungssprachen quantitativ gemessen werden können. Der Benchmark würde aus Referenzinformationssystemen verschiedener Domänen bestehen: einem eingebetteten Echtzeitsystem, einem verteilten Kommunikationsprotokoll, einem regelungstechnischen Regelkreis und einem sicherheitskritischen Medizinsystem. Für jedes Referenzsystem würde gemessen, wie vollständig, korrekt und wartbar die Beschreibung in der zu evaluierenden Sprache ausfällt.

10.2.3 Erweiterung um physikalische Phänomene

Das Ausgangszitat formuliert als Mindestanforderung, dass „möglichst jedes naturwissenschaftliche Phänomen oder Verhalten abgebildet werden kann“. Dies ist eine außergewöhnlich hohe Latte, die über klassische Software-Beschreibungssprachen hinausgeht. Zukünftige Forschung sollte untersuchen, wie Modelica [26] (für physikalische Systemsimulation) mit formalen Verifikationssprachen (PSL, LTL) zu einer hybriden Beschreibungssprache $\mathcal{L}_{\text{phys}}^*$ vereint werden kann, die sowohl kontinuierliche Dynamik als auch diskrete Steuerung vollständig erfasst.

10.3 Quantitative Theorie der Vollständigkeitslücke

Die in Satz 4 bewiesene Nichtleerheit der Vollständigkeitslücke $\Delta(\hat{\tau}_{CG}) \neq \emptyset$ ist qualitativ überzeugend und informationstheoretisch begründet, aber in ihrer quantitativen Ausprägung für konkrete Systeme noch nicht vollständig verstanden. Eine quantitative Theorie der Vollständigkeitslücke ist sowohl wissenschaftlich bedeutsam als auch industriell dringend benötigt.

10.3.1 Untere Schranken als Funktion der Systemkomplexität

Es sollen untere Schranken für $|\Delta(\hat{\tau}_{CG})|$ als Funktion der Systemkomplexität n abgeleitet werden. Die Arbeitshypothese lautet:

$$|\Delta(\hat{\tau}_{CG})(n)| \geq \Omega(n^\beta) \quad \text{für ein } \beta > 0,$$

d.h. die Vollständigkeitslücke wächst polynomiell in der Systemgröße. Dies würde bedeuten, dass für hinreichend große Systeme jedes Code-Generierungsverfahren eine superlinear wachsende Menge unerfüllter Anforderungen produziert – unabhängig von der Qualität

des Werkzeugs. Ein Beweis dieser Aussage würde Satz 4 wesentlich verschärfen und die Ablehnung von Code-Generierung für komplexe Systeme auf ein quantitatives Fundament stellen.

10.3.2 Experimentelle Messung in industriellen Werkzeugen

Parallel zur theoretischen Arbeit sind experimentelle Studien notwendig. Mit realen Modellierungswerkzeugen – MathWorks Simulink/Embedded Coder, IBM Rhapsody, ETAS ASCET, Elektrot EB tresos, ETAS ISOLAR-A – sollen konkrete Informationssysteme modelliert und die Code-Generierung anschließend durch den Subgraph-Verifikationsalgorithmus auf Vollständigkeit geprüft werden. Die Messungen würden erstmals empirische Belege für die in dieser Arbeit formal bewiesene Vollständigkeitslücke liefern.

10.3.3 Formale Charakterisierung durch temporale Logik

Formale Sprachen – insbesondere Hoare-Logik [27], die temporalen Logiken CTL [11] und LTL [12] sowie die modal μ -Kalkül – bieten präzise Mittel, um Vollständigkeitslücken zu charakterisieren. Eine Anforderung $r_i \in \mathcal{R}$ ist in LTL als eine Formel φ_i formulierbar; die Vollständigkeitslücke wird dann zur Menge

$$\Delta_{LTL}(\hat{\tau}_{CG}) := \{\varphi_i \in \Phi \mid \hat{\tau}_{CG}(G_{\mathcal{R}}) \not\models \varphi_i\}.$$

Zukünftige Forschung sollte diese Mengencharakterisierung für konkrete Systemklassen ausarbeiten und Werkzeuge entwickeln, die Δ_{LTL} automatisch berechnen.

10.4 Erweiterung des Subgraph-Algorithmus

10.4.1 Gewichtete und semantisch angereicherte Graphen

Der in Abschnitt 7 eingesetzte Subgraph-Algorithmus operiert auf ungewichteten Graphen mit binärer Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$. Reale Anforderungsgraphen sind jedoch häufig gewichtet und semantisch angereichert: Kanten tragen Prioritäten, Kosten, Zeitbeschränkungen, Wahrscheinlichkeiten oder Verfeinerungs-Grade. Eine natürliche Erweiterung der Subgraph-Signatur auf gewichtete Kanten lautet:

$$\sigma_j^w := \sum_{i=0}^{n-1} w_{ij} \cdot A_{ij} \cdot p^i + j \cdot p^n, \quad w_{ij} \in \mathbb{Q}_{>0}, \quad p \text{ prim},$$

wobei die Wahl einer Primzahl p als Basis die Injektivität in einem erweiterten Sinne sicherstellen kann (Chinesischer Restsatz). Diese Erweiterung erfordert jedoch eine neue vollständige Injektivitätstheorie, da der Beweis von Satz 6 auf der Binärkodierung basiert.

10.4.2 Hypergraphen für Multi-Stakeholder-Systeme

Informationssysteme in der Realwelt werden von mehreren Stakeholdern mit unterschiedlichen, teils konkurrierenden Anforderungen geprägt. Diese Situation lässt sich durch Hypergraphen modellieren, bei denen eine Hyperkante $e \subseteq V$ eine Anforderung kodiert, die

mehr als zwei Module betrifft. Die Erweiterung des Subgraph-Algorithmus auf Hypergraphen ist ein offenes Forschungsproblem; die Signaturkonstruktion muss auf Hyperkanten verallgemeinert werden:

$$\sigma_e := \sum_{v \in e} 2^{\text{id}(v)} + |e| \cdot 2^n, \quad e \subseteq V.$$

Die Komplexität des verallgemeinerten Algorithmus ist zu bestimmen und gegen die Komplexität des ursprünglichen $O(n^3)$ -Algorithmus zu benchmarken.

10.4.3 Dynamische Graphen für evolvierte Informationssysteme

Informationssysteme unterliegen über ihren Lebenszyklus Veränderungen: Anforderungen werden hinzugefügt, modifiziert oder entfernt. Entsprechend ist der Anforderungsgraph $G_{\mathcal{R}}$ kein statisches Objekt, sondern ein dynamischer Graph $G_{\mathcal{R}}(t)$, der sich über die Zeit verändert. Zukünftige Forschung sollte eine *inkrementelle* Variante des Subgraph-Algorithmus entwickeln, die bei einer lokalen Änderung δG nur die betroffenen Signaturen neu berechnet, statt den gesamten Algorithmus neu auszuführen. Die Zielsetzung wäre:

$$T_{\text{inkr}}(\delta G) = O(|\delta G| \cdot n^2) \ll O(n^3) = T_{\text{voll}}.$$

10.4.4 Parallelisierung und GPU-Beschleunigung

Der Subgraph-Algorithmus hat eine Zeitkomplexität von $O(n^3)$, die für sehr große Informationssysteme ($n > 10^4$ Knoten) in der Praxis zu langen Laufzeiten führt. Die n zyklischen Rotationen sind vollständig unabhängig voneinander (vgl. [22]) und eignen sich daher hervorragend für Parallelisierung:

$$T_{\text{parallel}}(n, p) = O\left(\frac{n^3}{p}\right) + O(n^2 \log p),$$

wobei p die Anzahl der Prozessoren und $O(n^2 \log p)$ der Kommunikationsaufwand ist. Eine GPU-beschleunigte Implementierung mittels CUDA [28] oder OpenCL könnte den Algorithmus für industrielle Echtzeit-Verifikationsszenarien mit mehreren tausend Modulen nutzbar machen. Für eingebettete Systeme mit begrenzten Ressourcen sind approximative Varianten zu untersuchen: Sampling einer repräsentativen Teilmenge der n Rotationen mit kontrollierter Fehlerrate, oder randomisierte LCS-Schätzungen auf Basis von Locality-Sensitive Hashing [29].

10.5 Integration in modellbasierte Entwicklungsprozesse (MBSE)

Model-Based Systems Engineering (MBSE) [13] ist der industrielle Standard für die Entwicklung komplexer Systeme und sieht explizit die Verwendung formaler Modelle auf allen Entwicklungsebenen vor. Die in dieser Arbeit entwickelten formalen Grundlagen – Anforderungsgraph $G_{\mathcal{R}}$ (Definition 4), Ingenieuroptimum (Definition 10), Vollständigkeitslücke (Definition 11) und Subgraph-Verifikation (Satz 7) – können direkt in MBSE-Werkzeuge und -Prozesse integriert werden.

10.5.1 SysML-Integration

SysML [14] ist die führende Modellierungssprache im MBSE-Kontext. Zukünftige Arbeit sollte einen SysML-Stereotyp `«InformationEngineer»` entwickeln, der die in Definition 15 spezifizierten Kompetenzkomponenten (K, M, L, R, T) als Tagged Values kodiert. Weiterhin sollte das Konzept des Anforderungsgraphen $G_{\mathcal{R}}$ als formale Erweiterung des Requirements-Diagramms in SysML spezifiziert werden, mit automatischer Subgraph-Konsistenzprüfung als integrierten Constraint-Block.

10.5.2 Werkzeugintegration

Der Subgraph-Algorithmus sollte als automatisches Verifikationsplugin in führende MBSE-Werkzeuge integriert werden: Sparx Enterprise Architect, Siemens Teamcenter, PTC Integrity Modeler, Capella (Thales). Das Plugin würde bei jeder Modellierung eines Implementierungsgraphen automatisch prüfen, ob $G_{\mathcal{R}} \subseteq G_{\mathcal{I}}$ gilt, und fehlende Anforderungserfüllungen direkt im Modell visualisieren.

10.5.3 Verbindung zu Digital Twins

Die Technologie des *Digital Twin* [33] – digitale Zwillinge physischer Systeme – ist eine natürliche Anwendungsdomäne für das in dieser Arbeit entwickelte Rahmenwerk. Der Anforderungsgraph $G_{\mathcal{R}}$ des Digital Twin sollte kontinuierlich mit dem Implementierungsgraph $G_{\mathcal{I}}$ des realen Systems synchronisiert und durch den Subgraph-Algorithmus auf Konsistenz geprüft werden. Dies würde die Vollständigkeit des Digital Twin formal garantieren – ein Forschungsziel von großer industrieller Relevanz.

10.6 Kognitionswissenschaftliche Fundierung der Erlernbarkeit

Die in Definition 8 eingeführte Erlernbarkeitsfunktion $\text{Learn}(\mathcal{L})$ ist bisher phänomenologisch motiviert. Eine tiefere kognitionswissenschaftliche Fundierung ist sowohl für die Ausbildung von Informationsingenieuren als auch für die Konstruktion der optimalen Beschreibungssprache $\mathcal{L}^*$ von unmittelbarer Relevanz.

10.6.1 Cognitive Load Theory

Die Cognitive Load Theory (CLT) nach Sweller [15] unterscheidet drei Arten kognitiver Last: intrinsische Last (inhärente Komplexität des Materials), extrinsische Last (durch schlechtes Lehrdesign erzeugte Mehrbelastung) und lernbezogene Last (aktiver Erwerb neuer Schemata). Für die Beschreibungssprache $\mathcal{L}^*$ folgt daraus: Die intrinsische Last ist durch die Ausdrucksmächtigkeit der Sprache bestimmt und kann nicht reduziert werden; die extrinsische Last kann durch klare Syntaxdefinitionen und gute Werkzeugunterstützung minimiert werden. Die Optimierung von $\text{Learn}(\mathcal{L}^*)$ reduziert sich damit auf die Minimierung der extrinsischen Last bei gegebener intrinsischer Last.

10.6.2 ACT-R-Theorie und prozedurales Lernen

Die ACT-R-Theorie [16] modelliert den Erwerb prozeduralen Wissens (Skills) als Produktion von Regeln im Langzeitgedächtnis. Für Beschreibungssprachen relevant ist die Beobachtung, dass prozedurale Kompetenz in einer Sprache exponentiell mit der Übungszeit

wächst (Power Law of Practice). Zukünftige Forschung sollte empirisch messen, nach welcher Übungszeit Ingenieure $\mathcal{L}^*$ produktiv einsetzen, und das ACT-R-Modell kalibrieren.

10.6.3 Interdisziplinäre Forschungsfrage

Die übergeordnete Forschungsfrage lautet: *Welche Syntaxmerkmale einer Beschreibungssprache maximieren die Lerngeschwindigkeit bei gegebener Ausdrucksmächtigkeit?* Diese Frage verbindet Informatik, Linguistik und kognitive Psychologie und sollte durch kontrollierte Experimente mit Ingenieuren verschiedener Ausbildungshintergründe untersucht werden.

10.7 Formale Semantik für Mehrhierarchie-Graphmodelle

Das Ausgangszitat betont ausdrücklich, dass Anforderungen „in beliebig vielen Hierarchien“ gegeben werden. Die in Definition 5 eingeführte hierarchische Graphzerlegung ist syntaktisch klar, aber ihre formale Semantik – insbesondere das Refinement-Verhältnis zwischen benachbarten Hierarchieebenen und die Konsistenzprüfung über mehrere Ebenen hinweg – bedarf weitergehender formaler Behandlung.

10.7.1 Vertikale Konsistenz

Eine vollständige vertikale Konsistenzaussage müsste zeigen, dass für eine hierarchische Zerlegung $G_0 \supseteq G_1 \supseteq \dots \supseteq G_d$ gilt:

1. Jedes G_{k+1} ist ein korrektes Refinement von G_k , d.h. keine Anforderungen aus G_k gehen bei der Verfeinerung verloren.
2. Die Verifikation auf Ebene d (Implementierungsebene) impliziert die Verifikation auf allen höheren Ebenen.

Ansätze aus der Prozessalgebra – CSP [17] mit seinem Refinement-Kalkül und CCS [18] mit Bisimulation – bieten einen natürlichen Ausgangspunkt. Die kategorientheoretische Sichtweise (Funkoren zwischen Kategorien von Graphen) ermöglicht eine elegante abstrakte Formulierung.

10.7.2 Horizontale Konsistenz

Neben der vertikalen Konsistenz ist die horizontale Konsistenz auf einer Hierarchieebene zu sichern: Zwei Subgraphen G_a und G_b , die verschiedene Subsysteme auf derselben Ebene beschreiben, dürfen keine widersprüchlichen Anforderungen kodieren. Die formale Charakterisierung von Widerspruchsfreiheit in einem Graphmodell – etwa durch Satisfiability von assoziierten Constraint-Systemen – ist ein weiteres offenes Problem.

10.8 Sicherheit, Zertifizierung und rechtliche Verankerung

10.8.1 Normkonforme Integration

Informationssysteme in sicherheitskritischen Umgebungen (Automobil, Medizintechnik, Luftfahrt, Kernenergie) unterliegen streng regulierten Normen. Die formale Verifikationskette aus Korollar 2 bietet eine natürliche Grundlage für normkonforme Entwicklungsprozesse:

- **ISO 26262 [19] (Automotive):** Der Anforderungsgraph $G_{\mathcal{R}}$ liefert die formale Basis für die Item Definition und die Hazard Analysis and Risk Assessment (HARA). Der Subgraph-Algorithmus kann die vollständige Anforderungsüberdeckung (Coverage Analysis) für jedes Safety Integrity Level (ASIL A bis D) nachweisen.
- **IEC 61508 [6] (allgemeine funktionale Sicherheit):** Die Vollständigkeitsgarantie $\Delta(\tau) = \emptyset$ entspricht der Forderung nach vollständiger Anforderungsüberdeckung auf Safety Integrity Level (SIL) 1 bis 4.
- **DO-178C [20] (Avionik):** Die formale Beschreibungssprache $\mathcal{L}^*$ kann die in DO-178C geforderten Software Level A–E unterstützen, wenn die Verifikation durch den Subgraph-Algorithmus als autorisiertes Verifikationswerkzeug anerkannt wird.
- **IEC 62443 [30] (Industrial Cybersecurity):** Die Transformationsverantwortung des Informationsingenieurs (Satz 3) umfasst die Sicherstellung der Security Requirements aus IEC 62443 in der Beschreibung $\mathcal{L}^*$.

10.8.2 Rechtliche Verankerung des Berufsbilds

Die Transformationsverantwortung des Informationsingenieurs hat nicht nur technische, sondern auch rechtliche Implikationen. Analog zum Bauingenieur, der für die Statik seines Bauwerks persönlich haftet, sollte der Informationsingenieur für die Korrektheit seiner Transformation τ und die Vollständigkeit seiner Verifikation haften. Zukünftige rechtswissenschaftliche Forschung sollte klären, welche Haftungsrahmen für Informationsingenieure angemessen sind, und ob eine formale Zulassungsprüfung (analog zur Architektenkammer) eingeführt werden sollte.

10.9 Informationsingenieurwesen und Künstliche Intelligenz

10.9.1 KI als Assistent in der Verifikationskette

Aktuelle Entwicklungen in der KI – insbesondere Large Language Models (LLMs) und neuronale Programmsynthese [31] – eröffnen neue Möglichkeiten in der Verifikationskette des Informationsingenieurs. LLMs können als Assistent bei der Konstruktion von $G_{\mathcal{R}}$ aus natürlichsprachlichen Anforderungen eingesetzt werden; die formale Verifikation durch den Subgraph-Algorithmus bleibt jedoch unersetzlich, da LLMs keine formalen Vollständigkeitsgarantien liefern können.

10.9.2 Grenzen der KI-gestützten Code-Generierung

Aus Satz 5 folgt unmittelbar, dass KI-gestützte Code-Generierung – etwa durch GitHub Copilot oder OpenAI Codex – keine Ausnahme von der Vollständigkeitslücke $\Delta \neq \emptyset$ darstellt. Die KI-generierte Code-Generierung ist eine spezielle Instanz von $\hat{\tau}_{CG}$ und unterliegt damit dem Beweis von Satz 4. Zukünftige Forschung sollte empirisch messen, wie groß die Vollständigkeitslücke KI-generierter Software im Vergleich zu manuell beschriebener Software ist – eine Frage von unmittelbarer praktischer Relevanz für den Einsatz von KI in sicherheitskritischen Informationssystemen.

10.9.3 Neurosymbolische Integration

Ein vielversprechender Forschungsansatz ist die neurosymbolische Integration [32]: die Verbindung von lernenden neuronalen Netzen (für die Extraktion von $G_{\mathcal{R}}$ aus unstrukturierten Anforderungsdokumenten) mit symbolischen Verifikationsverfahren (dem Subgraph-Algorithmus für die formale Konsistenzprüfung). Diese Kombination würde die Stärken beider Paradigmen vereinen und könnte die Konstruktion von $G_{\mathcal{R}}$ aus großen Mengen natürlichsprachlicher Anforderungen erheblich beschleunigen.

10.10 Anthropologische, gesellschaftliche und ethische Dimension

10.10.1 Informationssysteme für das Leben

Das Ausgangszitat benennt das höchste Ziel des Informationsingenieurs: „den Menschen Informationssysteme für das Leben zu geben“. Diese Formulierung ist nicht rein technisch, sondern zutiefst anthropologisch: Informationssysteme sind keine Selbstzwecke, sondern Dienste an menschlichen Bedürfnissen, Fähigkeiten und Würden. Die Frage, welche Informationssysteme dem Menschen *wirklich* dienen, ist nicht durch Algorithmen beantwortbar, sondern erfordert ethisch-philosophische Reflexion.

10.10.2 Technikethik als integraler Bestandteil des Berufsbilds

Zukünftige Forschung an der Schnittstelle von Informationsingenieurwesen und Technikethik sollte Kriterien entwickeln, nach denen der Informationsingenieur die Menschen dienlichkeit seines Systems beurteilen kann. Dabei sind folgende Fragen leitend:

- Welche Anforderungen $r_i \in \mathcal{R}$ kodieren ethische Grundprinzipien (Nicht-Schaden, Autonomie, Gerechtigkeit, Nutzen) und wie werden sie in $G_{\mathcal{R}}$ formalisiert?
- Wie kann die Vollständigkeitslücke Δ für ethische Anforderungen minimiert werden, wenn diese schwer formalisierbar sind?
- Welche Rolle spielt der Informationsingenieur bei der gesellschaftlichen Folgenabschätzung seines Systems?

10.10.3 Digitale Souveränität und Zugänglichkeit

Eine gesellschaftlich relevante Dimension des Informationsingenieurwesens ist die digitale Souveränität: Informationssysteme sollen nicht nur technisch korrekt sein, sondern auch zugänglich, verständlich und kontrollierbar für die Menschen, die sie benutzen. Dies stellt zusätzliche Anforderungen an die graphische Modellsicht – sie muss nicht nur für technische Entscheidungsträger, sondern auch für Bürger und Nutzer verständlich sein – und an die Beschreibungssprache, die auditierbar und transparent sein muss.

10.11 Langfristige Vision: Der Informationsingenieur als Grundberuf der Informationsgesellschaft

Alle in diesem Ausblick skizzierten Forschungsfelder konvergieren in einer langfristigen Vision: Der Informationsingenieur soll in der Informationsgesellschaft des 21. Jahrhunderts die Rolle einnehmen, die der Bauingenieur in der Industriegesellschaft des 19. Jahrhunderts

einnahm – als derjenige, der die physische Infrastruktur der Gesellschaft mit Verantwortung, Formalismus und Menschendienlichkeit gestaltet hat.

Die Informationsinfrastruktur des 21. Jahrhunderts – Kommunikationsnetze, Steuerungssysteme, medizinische Informationssysteme, autonome Fahrzeuge, smarte Städte – ist ebenso grundlegend für menschliches Leben wie Brücken, Gebäude und Wasserversorgung. Ihre Gestaltung erfordert einen Beruf, der die theoretischen Grundlagen des Informatikers und die praktische Kompetenz des Softwareentwicklers in einem einzigen kohärenten Berufsbild vereint – mit voller Verantwortung für die Transformation von der Anforderung zum Zielcode und mit dem klaren Ziel, den Menschen Informationssysteme für das Leben zu geben.

Dieser Beruf ist der Informationsingenieur.

Literatur

- [1] K. Sugiyama, S. Tagawa und M. Toda. *Methods for Visual Understanding of Hierarchical System Structures*. IEEE Transactions on Systems, Man, and Cybernetics, 11(2):109–125, 1981.
- [2] T. M. J. Fruchterman und E. M. Reingold. *Graph Drawing by Force-Directed Placement*. Software: Practice and Experience, 21(11):1129–1164, 1991.
- [3] A. M. Turing. *On Computable Numbers, with an Application to the Entscheidungsproblem*. Proceedings of the London Mathematical Society, 42:230–265, 1936.
- [4] H. G. Rice. *Classes of Recursively Enumerable Sets and Their Decision Problems*. Transactions of the American Mathematical Society, 74(2):358–366, 1953.
- [5] IEEE. *IEEE Standard 1220: Standard for Application and Management of the Systems Engineering Process*. IEEE, 2021.
- [6] IEC. *IEC 61508: Functional Safety of Electrical/Electronic/Programmable Electronic Safety-related Systems*. International Electrotechnical Commission, 2010.
- [7] IEEE. *IEEE Standard for Standard SystemC Language Reference Manual (IEEE 1666-2011)*. IEEE, 2012.
- [8] ISO. *ISO 8807: Information Processing Systems – Open Systems Interconnection – LOTOS – A Formal Description Technique Based on the Temporal Ordering of Observational Behaviour*. ISO, 1988.
- [9] IEEE. *IEEE Standard for Property Specification Language (IEEE 1850-2005)*. IEEE, 2005.
- [10] ITU-T. *Z.100: Specification and Description Language SDL*. ITU-T, 2000.
- [11] E. M. Clarke und E. A. Emerson. *Design and Synthesis of Synchronization Skeletons Using Branching-Time Temporal Logic*. Lecture Notes in Computer Science, 131:52–71, 1982.
- [12] A. Pnueli. *The Temporal Logic of Programs*. Proceedings of the 18th Annual Symposium on Foundations of Computer Science, pp. 46–57, 1977.

- [13] S. Friedenthal, A. Moore und R. Steiner. *A Practical Guide to SysML: The Systems Modeling Language*. Morgan Kaufmann, 2011.
- [14] OMG. *OMG Systems Modeling Language (OMG SysML) Version 1.6*. Object Management Group, 2019.
- [15] J. Sweller. *Cognitive Load During Problem Solving: Effects on Learning*. Cognitive Science, 12(2):257–285, 1988.
- [16] J. R. Anderson. *Rules of the Mind*. Lawrence Erlbaum Associates, 1993.
- [17] C. A. R. Hoare. *Communicating Sequential Processes*. Communications of the ACM, 21(8):666–677, 1978.
- [18] R. Milner. *A Calculus of Communicating Systems*. Lecture Notes in Computer Science, vol. 92. Springer, 1980.
- [19] ISO. *ISO 26262: Road Vehicles – Functional Safety*. International Organization for Standardization, 2018.
- [20] RTCA. *DO-178C: Software Considerations in Airborne Systems and Equipment Certification*. RTCA, 2011.
- [21] A. Abboud, A. Backurs und V. Vassilevska Williams. *Tight Hardness Results for LCS and Other Sequence Similarity Measures*. Proceedings of the 56th Annual Symposium on Foundations of Computer Science (FOCS), pp. 59–78, 2015.
- [22] S. Epp. *Der Subgraph Algorithmus: Effiziente Graphverifikation mittels Signatur-Arrays und zyklischer Rotation*. Universität Bielefeld, 2026. <https://gitlab.com/epp-group/subgraph>
- [23] C. E. Shannon. *A Mathematical Theory of Communication*. The Bell System Technical Journal, 27(3):379–423, 1948.
- [24] INCOSE. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 4th Edition. International Council on Systems Engineering, Wiley, 2015.
- [25] SAE International. *AS5506: Architecture Analysis and Design Language (AADL)*. SAE International, 2004.
- [26] Modelica Association. *Modelica – A Unified Object-Oriented Language for Systems Modeling, Language Specification Version 3.3*. Modelica Association, 2014.
- [27] C. A. R. Hoare. *An Axiomatic Basis for Computer Programming*. Communications of the ACM, 12(10):576–580, 1969.
- [28] NVIDIA Corporation. *CUDA C++ Programming Guide*. NVIDIA, 2023.
- [29] P. Indyk und R. Motwani. *Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality*. Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC), pp. 604–613, 1998.

- [30] IEC. *IEC 62443: Industrial Communication Networks – Network and System Security*. International Electrotechnical Commission, 2018.
- [31] M. Chen et al. *Evaluating Large Language Models Trained on Code*. arXiv preprint arXiv:2107.03374, 2021.
- [32] L. C. Lamb, A. Garcez, M. Gori, M. Prates, P. Avelar und M. Vardi. *Graph Neural Networks Meet Neural-Symbolic Computing: A Survey and Perspective*. Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI), pp. 4877–4884, 2020.
- [33] M. Grieves. *Digital Twin: Manufacturing Excellence Through Virtual Factory Replication*. White Paper, Florida Institute of Technology, 2014.